Ain Shams University
Faculty of Computer & Information Sciences
Computer Science Department

Doc2Pod
Arabic Podcast Generator

By:

Esraa Said Shaban
Fatma Mahmoud Abdelkader
Laila Mohammed Abo El-Fotouh
Mohammed Atef Moselhy
Mohammed Ehab Hussein
Yousef Ali Omar

Under Supervision of :

Dr. Sally Saad
Social Professor of Computer Science,
Computer Science Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

TA. Mayar Mohammed
Teaching Assistant,
Information Systems Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

Acknowledgement

First and foremost, we express our deepest gratitude to Allah Almighty for granting us the strength, patience, wisdom, and perseverance to complete this project. His endless blessings, guidance, and mercy have been our greatest source of motivation throughout this journey. Without His support, this achievement would not have been possible.

We would like to extend our sincere and heartfelt appreciation to **Dr. Sally Saad** for her exceptional supervision, continuous guidance, and unwavering encouragement throughout every stage of this project. Her invaluable expertise, insightful feedback, thoughtful recommendations, and constant support have played a fundamental role in shaping the quality of this work. We are truly grateful for the time, effort, and confidence she invested in us, which inspired us to overcome challenges and continuously improve our research and implementation.

Our sincere thanks also go to **TA Mayar Mohamed** for her remarkable dedication, patience, and continuous assistance throughout the development of this project. Her constructive comments, technical guidance, and willingness to provide support whenever needed significantly contributed to our progress. Her encouragement and valuable suggestions helped us refine our ideas and maintain steady progress throughout the project.

We would also like to express our gratitude to the **Faculty of Computer and Information Sciences, Ain Shams University**, for providing an outstanding academic environment that fostered learning, creativity, and innovation. The knowledge, resources, facilities, and educational opportunities provided by the faculty have been instrumental in developing the technical and research skills required to accomplish this work.

Finally, we would like to extend our appreciation to everyone who contributed to the success of this project, whether directly or indirectly. Every piece of advice, encouragement, and support, no matter how small, has been deeply valued and has contributed to making this work possible.

We hope that this project serves as a meaningful contribution to the field and reflects the knowledge, dedication, and collaborative efforts invested throughout its development.

Doc2Pod Team

Abstract

Computer Science students are frequently required to study extensive volumes of technical educational materials, including textbooks, research papers, lecture notes, software documentation, and online technical resources. While these materials are essential for building both theoretical knowledge and practical expertise, they are often characterized by complex terminology, dense explanations, and lengthy content. As a result, students may find it difficult to efficiently absorb information, remain engaged throughout the learning process, or dedicate sufficient time to reading large amounts of technical material. Meanwhile, audio-based learning has gained widespread popularity as a flexible and accessible educational approach, enabling learners to consume content while commuting, exercising, or performing other daily activities.

This project presents **Doc2Pod**, an AI-powered educational system designed to transform technical documents into engaging podcast-style audio specifically tailored for Computer Science students. The system accepts PDF documents and text-based educational content as input, extracts and analyzes their structure and semantic content, and generates educational conversations in Egyptian Arabic while preserving English technical terminology to maintain technical accuracy and familiarity. The generated scripts are then converted into natural-sounding speech, producing high-quality educational podcasts that provide an alternative and more accessible learning experience.

To accomplish this objective, Doc2Pod integrates multiple state-of-the-art artificial intelligence technologies within a unified processing pipeline. These include advanced document understanding models, Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), and modern neural Text-to-Speech (TTS) models. The system is also evaluated using automated metrics and qualitative assessment to measure the quality, coherence, technical accuracy, and naturalness of the generated podcasts.

The proposed solution aims to improve the accessibility of technical educational materials, increase learner engagement, and reduce the effort required to consume complex content. By leveraging recent advances in artificial intelligence, Doc2Pod provides an effective and flexible learning experience that complements traditional reading-based study methods.

Table of Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Stands For
AI	Artificial Intelligence
CMD	Content Detection Metric
GQA	Grouped-Query Attention
ICASSP	International Conference on Acoustics, Speech, and Signal Processing
KV	Key-Value
LateX	Lamport TeX Document Preparation System
LLM	Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
RoPE	Rotary Position Embedding
RT-DETR	Real-Time Detection Transformer
SOTA	State of the Art
STEM	Science, Technology, Engineering, and Mathematics
TEDS	Tree Edit Distance-based Similarity
TTS	Text-to-Speech
VCTK	Centre for Speech Technology Voice Cloning Toolkit Dataset
VITS	Variational Inference with Adversarial Learning for End-to-End Text-to-Speech
VLM	Vision-Language Model

1- Introduction

1.1 Motivation

The idea for this project was born from a deeply frustrating reality we and countless computer science students face: staring at essential textbooks or research papers, knowing we must read them, yet feeling our minds resist with every page—not from laziness, but from the suffocating weight of dense, monotonous text that drains energy before we even begin.

We've all experienced opening a PDF at midnight, reading the same paragraph three times without absorbing anything, then closing it in defeat, while guilt piles up alongside unread chapters and what should be exciting knowledge becomes a source of dread and procrastination.
Then we noticed something striking: we never feel this resistance with podcasts—we can listen for hours, captivated and learning, without noticing time pass. This contrast revealed the core problem: it's not the content; it's how we're forced to consume it.
This realization sparked our vision: a system that transforms academic content into podcast-style audio using Egyptian colloquial Arabic (the way we naturally think and communicate) while keeping technical terms like "binary search tree" or "gradient descent" in English to match how Egyptian CS students authentically code-switch.
By adding voices with real emotion—enthusiasm, emphasis, natural rhythm—rather than robotic text-to-speech, we create not just information delivery but a transformation of the entire learning experience. This project matters because it attacks a fundamental educational problem: format-induced disengagement, where thousands of students abandon valuable knowledge not because they can't understand it, but because traditional text makes the journey so painful they never start.

We're making content faster to consume and genuinely enjoyable to experience—cutting through research papers in half the time because students are actually focused, or finally finishing textbooks they've avoided for months because learning no longer feels like punishment.
In a field where staying current with rapid technological advancement is crucial but the volume of material is overwhelming, our system removes the psychological barrier between students and knowledge, transforming "I should read this" into "I want to hear this," making learning not just easier but possible at all.

1.2 Problem Definition

Computer Science students **rely on** a wide range of educational resources, including textbooks, research papers, lecture notes, and technical documentation. Although these resources contain valuable knowledge, they are often lengthy, highly technical, and difficult to consume efficiently. As a result, many students struggle to maintain focus while reading dense academic material, which may reduce learner engagement and negatively affect the learning process.

At the same time, modern learners increasingly prefer audio-based educational content because it enables them to learn while commuting, exercising, or performing other daily activities. However, most educational resources remain primarily text-based, while traditional text-to-speech solutions typically produce monotonous narration without simplifying concepts or adapting the content to support an effective learning experience.

Therefore, there is a need for a system that can automatically transform technical academic content into engaging educational podcasts that are easier to consume while **preserving the technical accuracy of the original content**.

1.3 Objective

The primary objective of this project is to **improve and simplify** the learning experience for Computer Science students by transforming complex technical content into engaging podcast-style educational content.

To achieve this objective, the project aims to:

- Support both **plain-text and PDF documents** as input sources to provide flexibility across different learning materials.
- Generate educational podcasts in **colloquial Egyptian Arabic** to reduce the need for lengthy reading sessions while improving accessibility.
- Preserve **English technical terminology** to maintain technical accuracy and clarity.
- Generate **structured, step-by-step explanations** with smooth transitions while anticipating common learner questions to enhance comprehension and learner engagement.

1.4 Time Plan

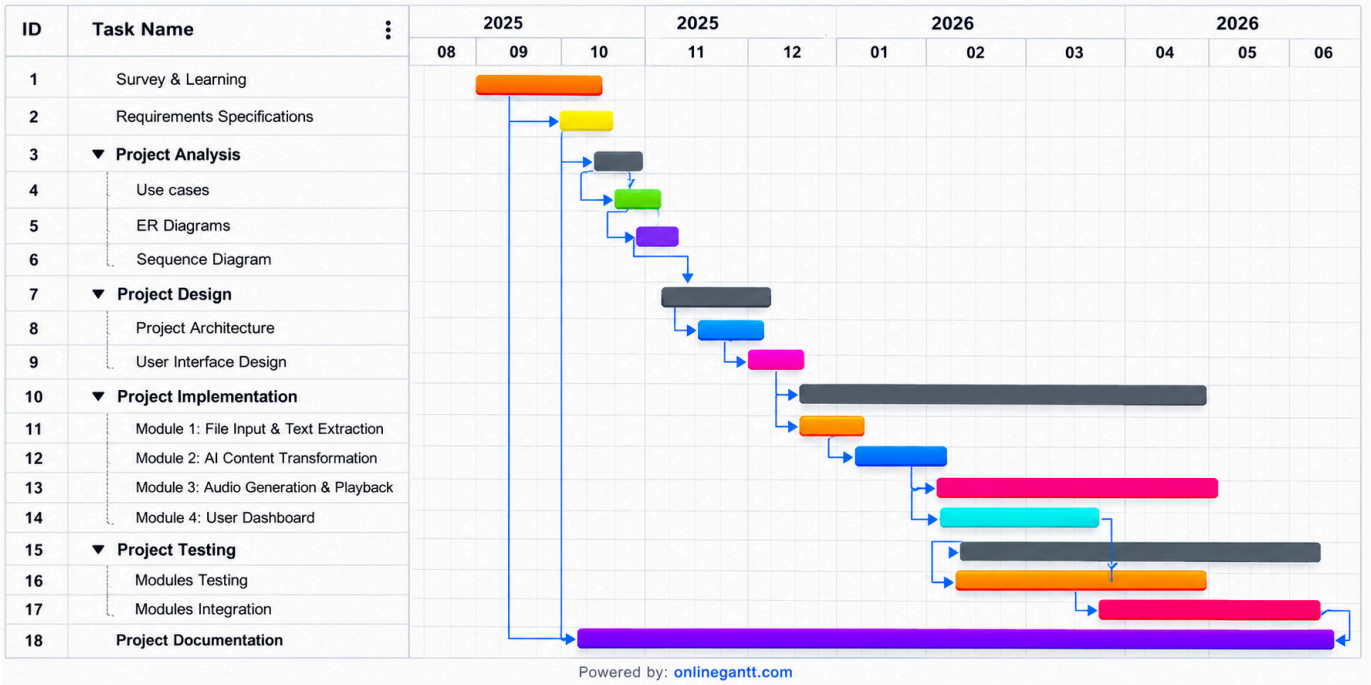

Figure 1.1: Gantt Chart of the Doc2Pod Project Timeline

1.5 Document Organization

This document is organized into six chapters.

Chapter 2 (Background) presents the scientific background of the project, discusses the related technologies, surveys existing solutions, and reviews previous research relevant to AI-powered podcast generation and speech synthesis.

Chapter 3 (Analysis and Design) describes the system architecture, user requirements, design diagrams, database structure, and overall system design.

Chapter 4 (Implementation and Testing) explains the implementation details, datasets, technologies, algorithms, user interface design, and experimental results.

Chapter 5 (User Manual) provides detailed instructions on how to install, configure, and use the system.

Chapter 6 (Conclusion and Future Work) summarizes the project outcomes, discusses the achieved objectives, and presents possible future enhancements

2- Background

This chapter provides a comprehensive review of the existing literature and prior work directly relevant to the Doc2Pod system. It is structured to first present a theoretical overview of the core technical fields that enable the system, followed by a detailed survey of related works across each major pipeline component. The chapter then draws a comparative analysis of these works, highlighting their strengths and limitations, which ultimately motivated the design decisions made in this project. The insights gathered from this review formed the foundation for addressing the identified gaps and building a system that is more cohesive, language-aware, and tailored to the needs of Egyptian Computer Science students.

2.1 Project Overview

The primary challenge addressed by Doc2Pod is the significant difficulty Egyptian Computer Science students face when engaging with dense, English-language academic materials such as lecture slides, textbooks, and research papers. Although audio-based learning has well-documented benefits in improving knowledge retention and reducing cognitive load, there remains a critical gap in tools capable of automatically transforming complex technical documents into natural, long-form, and culturally relevant conversational audio in Egyptian Arabic.

To bridge this gap, Doc2Pod integrates a comprehensive multi-stage AI pipeline that combines advanced document understanding, intelligent content retrieval, conversational script generation, and expressive dialect-specific speech synthesis.

Natural Language Processing (NLP) is the subfield of Artificial Intelligence concerned with enabling machines to understand, interpret, and generate human language. Early systems relied on rule-based and statistical methods. The introduction of the Transformer architecture in 2017 revolutionized the field by enabling effective modeling of long-range dependencies. Large Language Models (LLMs), pre-trained on massive text corpora, have since shown remarkable capabilities in summarization, reasoning, and dialogue generation with minimal task-specific training. In podcast generation, LLMs play a central role in transforming technical content into coherent and engaging conversational scripts.

Document Understanding focuses on extracting structured and semantically meaningful information from PDFs and scanned images. Traditional Optical Character Recognition (OCR) systems often struggle with complex academic layouts, tables, mathematical formulas, and figures. Modern approaches integrate layout analysis, table and chart recognition, and Vision-Language Models (VLMs) to achieve deeper understanding of both textual and visual content. This stage is essential for preserving accuracy when processing heterogeneous technical documents.

Retrieval-Augmented Generation (RAG) is a paradigm that improves the outputs of Large Language Models by grounding them in relevant external knowledge retrieved from a vector database. This approach helps mitigate hallucination and enhances factual consistency, especially when dealing with long or domain-specific documents. A typical RAG pipeline includes document chunking, dense embeddings, semantic retrieval, and context injection into the generation process.

Neural Text-to-Speech (TTS) synthesis is the task of generating natural-sounding spoken audio from text. The field has evolved from concatenative and statistical parametric methods to advanced end-to-end deep learning models capable of producing expressive speech with proper prosody and intonation. Recent research focuses on multi-speaker support, long-form generation, and adaptation to different languages and dialects.

2.2 Related Work

The Doc2Pod pipeline integrates four major technical components: document understanding and parsing, retrieval-augmented generation, LLM-based script generation, and neural text-to-speech synthesis. For each component, we surveyed the state-of-the-art systems and research, examined their methodologies, strengths, and limitations, and identified the gaps that motivated our design choices. The following subsections present these works organized by pipeline stage.

2.2.1 Document Understanding and Parsing

Document understanding and parsing is the foundational stage in any Document-to-Podcast system. Its quality directly determines the accuracy, completeness, and structure of the content fed to subsequent stages. Two prominent approaches in this area are PaddleOCR-VL and Nougat.

PaddleOCR-VL (Cui et al., 2025), developed by the Baidu PaddlePaddle team, is a state-of-the-art lightweight multimodal document parsing system. It follows a two-stage architecture: the PP-DocLayoutV2 module first performs layout analysis using RT-DETR for element detection and a pointer network for reading order prediction. The structured regions are then processed by a 0.9B-parameter Vision-Language Model (NaViT-style visual encoder + ERNIE-4.5-0.3B) for semantic understanding of text, tables, formulas, and charts. The model supports over 100 languages and was evaluated on OmniDocBench v1.5, achieving a new SOTA overall score of 92.86% on OmniDocBench v1.5, surpassing previous models and showing leading results in text recognition, formula parsing (CDM), and table structure understanding (TEDS). It outputs structured JSON or Markdown, making it highly suitable for downstream RAG pipelines. Its strengths lie in handling complex academic slides with mixed content, while its main limitation is the relatively higher computational cost during full inference compared to simpler OCR tools.

Nougat (Neural Optical Understanding for Academic Documents) (Blecher et al., 2024), developed by Meta AI, adopts an end-to-end Transformer-based approach for academic document parsing. The model converts full page images directly into structured markup (primarily LaTeX/Mathpix-style Markdown) using a Swin Transformer encoder for visual feature extraction and an mBART decoder for sequence generation. It was trained on a large-scale dataset of more than **1.75 million scientific papers from arXiv, comprising over 7.5 million pages**, paired with their original **LaTeX source files**, with a **particular focus on preserving complex mathematical notation**, is **fully supported by multiple explicit references** and data tables within the provided paper. Nougat demonstrates strong performance on formula-heavy STEM documents and outperforms traditional OCR systems in semantic structure preservation. However, it suffers from slower inference speed due to its end-to-end nature, limited optimization for multilingual or dense lecture slides, and the need for significant post-processing of the LaTeX output for general NLP tasks.

2.2.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) has become a critical component in modern document intelligence systems, addressing the hallucination problem of Large Language Models by grounding generated content in retrieved external knowledge. In the context of

Document-to-Podcast pipelines, effective RAG is essential for retrieving coherent, contextually relevant segments from complex technical documents while preserving semantic relationships across text, tables, and visuals. Two notable recent frameworks in this area are RAG-Anything and MDocAgent.

RAG-Anything (Guo et al., 2025) is a unified multimodal RAG framework designed to overcome the limitations of traditional text-only retrieval systems when handling heterogeneous documents. The framework introduces a dual-graph knowledge representation consisting of a text-based knowledge graph that captures entity-relation triples and a cross-modal knowledge graph linking textual entities to visual and tabular counterparts. During retrieval, it employs a hybrid strategy combining structural graph navigation with dense vector embeddings. The system was evaluated on two challenging multimodal benchmarks: DocBench and MMLongBench. DocBench contains 229 multimodal documents across diverse domains (Academia, Finance, Legal, etc.) with 1,102 expert-crafted QA pairs, while MMLongBench includes 135 long-context multimodal documents with 1,082 annotated questions. These benchmarks demonstrated the framework's superior performance over conventional text-only RAG baselines, particularly in cross-modal and long-context reasoning scenarios.

MDocAgent (Han et al., 2025) proposes a multi-agent collaborative framework to address the challenges of complex multimodal document understanding. The system consists of five specialized agents working in parallel: a General Agent responsible for overall coordination and reasoning, a Critical Agent that analyzes the query to identify required information, a Text Agent that performs dense retrieval from textual passages using ColBERT-style embeddings, an Image Agent that retrieves relevant visual regions using ColPali visual embeddings, and a Summary Agent that integrates outputs from all agents to produce a coherent final response. This parallelized multi-agent architecture enables effective handling of questions that require cross-modal reasoning, such as interpreting charts, tables, and their accompanying textual explanations simultaneously. The framework was evaluated on several multimodal document understanding benchmarks, where it achieved significant improvements over single-agent and traditional RAG baselines. However, the requirement to execute multiple specialized agents for each query leads to high computational cost and substantial inference latency. These limitations make MDocAgent less suitable for applications that demand frequent retrieval operations and low latency, such as long-form conversational content generation from dense academic materials.

2.2.3 Script Generation using Large Language Models

The transformation of structured technical content into engaging conversational scripts is a central challenge in Document-to-Podcast systems. Large Language Models have emerged as the dominant approach for this task due to their advanced instruction-following, reasoning, and multilingual generation capabilities. This section reviews three leading open-weight models evaluated for script generation: Qwen3, Gemma 3, and LLaMA 3.

Qwen3 (Yang et al., 2025) is the third generation of Alibaba Cloud's Qwen family of large language models, comprising both dense and Mixture-of-Experts (MoE) architectures with parameter sizes ranging from 0.6 billion to 235 billion parameters. The models were pre-trained on approximately 36 trillion tokens, covering a wide variety of domains, including scientific literature, programming resources and mathematical content. To further improve data quality and coverage, particularly in specialized domains and underrepresented languages, the training corpus was augmented with large-scale synthetic data generated by domain-specific teacher models.Among the released variants, **Qwen3-4B-Instruct-2507** offers a favorable balance between performance and computational efficiency. Despite its relatively small size, it achieves an MMLU-Pro score of 69.6 and demonstrates strong instruction-following and reasoning capabilities.

Gemma 3 (Gemma Team, 2024) is Google DeepMind's family of open-weight multimodal language models. The series includes models ranging from 1 billion to 27 billion parameters. A key architectural innovation in Gemma 3 is the shift to a 5:1 ratio of local to global attention layers, combined with RoPE rescaling, which enables efficient support for a 128K token context window while significantly reducing KV-cache memory usage during inference. The models were pre-trained on a large-scale multimodal dataset ranging from 4 trillion tokens for the 4B variant to 14 trillion tokens for the 27B variant. The pre-training data consists of a diverse mixture of web text, code, and multilingual corpora, further enhanced by the integration of image data processed through a frozen SigLIP vision encoder. To improve data quality, the team applied aggressive filtering to remove unsafe content and duplicates, along with quality reweighting techniques. On standard benchmarks, the **Gemma-3-IT-4B** variant achieves an MMLU-Pro score of 43.6, demonstrating strong instruction-following performance, particularly in long-context and multimodal tasks.

LLaMA 3 (Grattafiori et al., 2024) is Meta AI's third generation of open-weight large language models. The series includes models ranging from 8 billion to 405 billion parameters. A key architectural innovation in LLaMA 3 is the use of an optimized dense Transformer with Grouped-Query Attention (GQA) and a tokenizer vocabulary of 128K tokens, along with RoPE positional embeddings scaled to $\theta = 500{,}000$. The models were pre-trained on over 15 trillion tokens of publicly available multilingual data. The pre-training dataset features a carefully balanced mix emphasizing general knowledge, coding, reasoning, and multilingual content. To enhance data quality, the team applied rigorous filtering and quality ranking, particularly for non-English data. On standard benchmarks, the **LLaMA-3-8B** variant achieves an MMLU-Pro score of 48.3, demonstrating strong general language understanding and reasoning capabilities, though its performance on dialectal Arabic remains notably weaker than models with broader multilingual pre-training.

2.2.4 Neural Text-to-Speech Synthesis

PortaSpeech (Ren et al., 2021) is a non-autoregressive TTS system designed to address the limitations of blurry mel-spectrograms in flow-based models and unnatural prosody in duration-based models. It introduces a variational generator enhanced with linguistic features and a lightweight flow-based post-net to refine the generated mel-spectrograms. The model was trained on the LJSpeech dataset and achieves a Mean Opinion Score for quality (MOS-Q) of 3.92 and for prosody (MOS-P) of 3.89. These results are competitive with autoregressive baselines while offering significantly faster inference speed. However, PortaSpeech is a single-speaker model trained exclusively on English speech data. It lacks native support for multi-speaker synthesis, long-form generation, or Arabic phonemes, making it unsuitable for Egyptian Arabic conversational synthesis without major architectural changes.

YourTTS (Casanova et al., 2022**)** extends the VITS architecture to support zero-shot multilingual and multi-speaker speech synthesis. The system uses speaker embeddings from reference audio and language embeddings to condition the model, enabling synthesis in unseen voices and languages. It accepts raw text input and was trained on multilingual datasets including English (VCTK), French (M-AILABS), and Portuguese. The model achieves a MOS of approximately 4.20 and speaker similarity score near 0.864 on cross-lingual tasks. However, it suffers from unstable duration prediction on long utterances,

mispronunciations, and complete absence of Arabic data. Additionally, gender imbalance in the training data further affects synthesis quality across different speakers, making it unsuitable for Egyptian Arabic and code-switching without major modifications.

VibeVoice (Peng et al., 2025) is a long-form multi-speaker TTS system capable of generating up to 90 minutes of continuous conversational audio. The architecture combines a pre-trained LLM backbone (Qwen2.5) with a token-level diffusion head and uses separate semantic and acoustic tokenizers operating at an ultra-low frame rate of 7.5 Hz. It was evaluated on long conversational transcripts by 24 human annotators, achieving MOS scores of 3.71 for realism, 3.81 for richness, and 3.75 for overall preference, outperforming several commercial systems. While VibeVoice demonstrates exceptional long-form capability, speaker consistency, and natural prosody, the base model supports only English and Chinese. Therefore, it requires domain-specific fine-tuning on Egyptian Arabic data with proper code-switching patterns .

F5-TTS (Chen et al., 2025) is a fully non-autoregressive text-to-speech system based on Flow Matching with a Diffusion Transformer (DiT) backbone. Unlike conventional TTS architectures that rely on explicit duration models, phoneme alignment, or separate text encoders, F5-TTS simplifies the synthesis pipeline by padding text sequences with filler tokens and learning text-to-speech alignment directly. The model further incorporates a ConvNeXt-based text representation module and introduces Sway Sampling, an inference strategy that improves synthesis quality and efficiency without requiring retraining. Trained on approximately 100,000 hours of multilingual speech data, F5-TTS demonstrates high-quality zero-shot voice cloning, natural prosody, seamless code-switching, and fast inference with a reported real-time factor (RTF) of approximately 0.15. However, despite its multilingual capabilities, the released model does not explicitly support Egyptian Arabic and therefore still requires fine-tuning on suitable Arabic speech datasets to achieve accurate pronunciation and natural conversational synthesis for Egyptian Arabic applications.

2.2.5 End-to-End AI Podcast Generation Systems

End-to-end AI podcast generation systems aim to automate the full pipeline producing conversational audio directly suitable for listening. These systems represent the closest prior art to Doc2Pod and highlight both the progress and remaining gaps in handling technical documents

and low-resource dialects. Key representative works include PodAgent and MoonCast, alongside the prominent commercial system NotebookLM.

PodAgent (Xiao et al., 2025) introduces a multi-agent framework for generating talk-show-style podcasts from a user-provided topic. The system uses a Host-Guest-Writer architecture in which the Host-Agent creates guest profiles and a structured interview outline, multiple Guest-Agents generate diverse expert responses in parallel, and the Writer-Agent synthesizes them into a coherent conversational script. Voice pools were constructed from LibriTTS-R (English, 222 unique speakers after deduplication) and AISHELL-3 (Mandarin), with SpeechCraft annotations used for voice characteristic matching. CosyVoice2, guided by LLM-generated speaking style instructions, is employed for speech synthesis. The framework was evaluated on 40 topics and achieved notable improvements in dialogue diversity, informativeness, and 87.4% voice-role matching accuracy over GPT-4 baselines. However, PodAgent is limited because it depends solely on a topic name as input and does not accept or process existing documents as source content. Additionally, it only supports English and Mandarin, with no capability for Egyptian Arabic or code-switching. It also lacks advanced document understanding and retrieval mechanisms.

MoonCast (Ju et al., 2026**)** proposes a zero-shot two-speaker podcast generation system that converts text sources such as PDFs, articles, or web URLs into natural conversational audio. The system first uses an LLM (Gemini 2.0 Pro) to create a structured briefing document summarizing the input content, which then serves as input for script generation in JSON format containing speaker turns with spontaneous elements (fillers, pauses, and breathing cues). It employs a long-context audio language model trained on a large-scale dataset of approximately 300,000 hours of Chinese audiobooks and 200,000 hours of English conversational data. Experiments showed strong improvements in spontaneity and contextual coherence compared to cascaded baselines. However, MoonCast is currently limited to English and Chinese, supports only two-speaker conversations, relies on general document parsing , and lacks any support for Egyptian Arabic or technical code-switching.

Google NotebookLM – Audio Overview (Google, 2024–2026) is a prominent commercial system developed by Google that automatically converts uploaded documents, PDFs, lecture slides, and web content into conversational two-host podcasts. Powered by Gemini models with long-context capabilities, the system analyzes the input material and generates natural dialogue between two AI hosts (typically one male and one female) in an engaging discussion style. It supports multiple output modes such as "Deep Dive" and "Brief," and offers multilingual capabilities including Arabic. The system benefits from Google's large-scale multimodal pre-training and advanced neural TTS for natural prosody and turn-taking. While NotebookLM excels in ease of use and producing listenable conversational audio for general content, it has several limitations: it applies heavy summarization on long or dense technical documents, offers limited user control over content depth or section selection, uses fixed host voices, and faces daily generation quotas even on paid tiers.

2.3 Comparative Analysis

To provide a clearer understanding of the current landscape, a component-based comparative analysis of the reviewed systems is presented in the following tables. This comparison focuses on the four main stages of the Doc2Pod pipeline and highlights the strengths, limitations, and gaps that the proposed system aims to address.

Table 2.1: Comparison of Leading Document Understanding Approaches

Model/Aspect	Architecture	Primary Strength	Benchmark/ Dataset	Output Format
PaddleOCR-VL (Cui et al., 2025)	Two-stage (Layout Analysis + 0.9B VLM)	Tables, charts, reading order, multilingual	OmniDocBench v1.5	Structured JSON / Markdown
Nougat (Blecher et al., 2024)	End-to-End Transformer	Mathematical formulas & LaTeX	Strong on arXiv formula preservation	LaTeX / Markdown

Table 2.2: Comparison of LLMs for Script Generation

Model	MMLU-Pro	Context Window	IFEval
Qwen3-4B-Instruct-2507 (Yang et al., 2025)	69.6	256K	83.4
Gemma-3-IT-4B (Gemma Team, 2024)	43.6	128K	90.2
LLaMA-3-8B (Grattafiori et al., 2024)	48.3	8K	80.4

Table 2.3: Comparison of Text-to-Speech Systems

Model	Metrics
PortaSpeech (Ren et al., 2021)	MOS-Q: 3.92 MOS-P: 3.89
YourTTS (Casanova et al., 2022)	MOS: 4.20 Sim-MOS: 4.17 Spk SIM: 0.864
VibeVoice-7B (Peng et al., 2025)	MOS (Pref): 3.75 MOS (Real): 3.81 MOS (Rich): 3.77 S-SIM: 0.692

Based on the background and related systems discussed in this chapter , several limitations were identified across existing Document-to-Podcast systems. Existing document parsing methods either introduce high computational overhead or require extensive post-processing for downstream NLP applications. Furthermore, existing LLMs and TTS systems provide limited support for Egyptian Arabic, technical terminology, and code-switching between Arabic and English.

Most importantly, current end-to-end podcast generation systems are primarily designed for general-purpose content in English and Chinese and do not adequately address the needs of Egyptian Computer Science students. They often heavily summarize technical documents, provide limited control over content depth, and lack support for generating educational podcasts that preserve technical details while using natural Egyptian Arabic conversational style.

These limitations motivated the design of Doc2Pod. The proposed system integrates advanced multimodal document understanding and retrieval techniques, generates conversational scripts tailored to technical educational content, and incorporates neural speech synthesis adapted for Egyptian Arabic and code-switching scenarios. By focusing specifically on the needs of Egyptian Computer Science students, Doc2Pod aims to provide an end-to-end solution that transforms complex academic documents into accessible, engaging, and educational podcast-style audio content.

3- Analysis and Design

3.1 System Overview

3.1.1 System Architecture

Doc2Pod is an AI-powered platform that transforms PDF documents into natural-sounding Egyptian Arabic podcasts. The system combines modern web technologies, Natural Language Processing (NLP), Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), and Text-to-Speech (TTS) technologies to provide an end-to-end automated content generation pipeline.

Users can upload PDF documents through a web interface, track processing progress in real time, and listen to or download the generated podcast. The platform extracts and understands document content, generates a conversational podcast script in Egyptian Arabic, and finally converts the script into high-quality audio.

The architecture follows a layered design that promotes **scalability, maintainability, separation of concerns,** and **efficient processing of large documents.** Figure below illustrates the overall system architecture and the interaction between its major components.

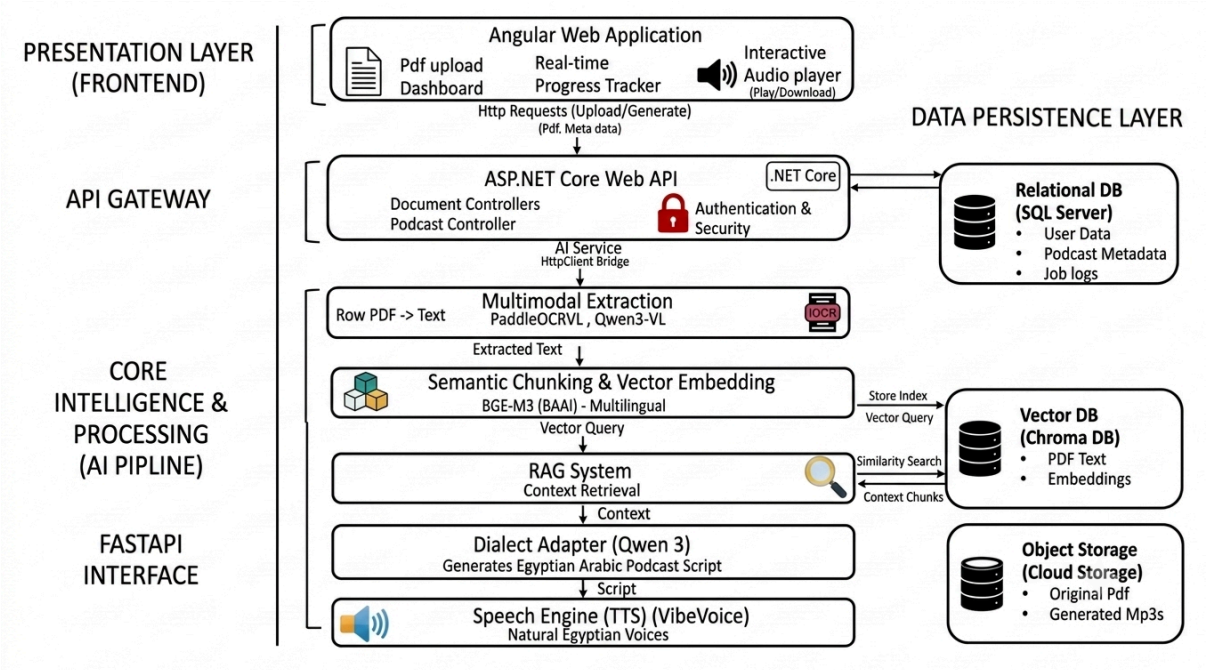

Figure 3.1.1.1 System architecture

The system is divided into **four** major layers:

1. Presentation Layer (Frontend)
2. Application Layer (API Gateway)
3. Core Intelligence & Processing Layer (AI Pipeline)
4. Data Persistence Layer

1. Presentation Layer [Frontend]

The Presentation Layer represents the user-facing interface of the system and is implemented as an **Angular Web Application**. Its primary responsibility is to provide a seamless and interactive experience for users throughout the document-to-podcast generation process.

Responsibilities:

- Provide user authentication and account management functionalities.
- Handle PDF document uploads.
- Manage user interactions and navigation.
- Play generated podcasts directly in the browser.
- Allow users to download generated audio files.
- Provide access to previously generated podcasts and document history.

2. Application Layer [API Gateway]

The Application Layer serves as the central communication bridge between the frontend and the backend processing services. It is implemented using **ASP.NET Core Web API** and exposes RESTful endpoints for handling all client requests.

Responsibilities:

- Handle authentication and authorization.
- Receive and validate incoming HTTP requests.
- Manage document upload operations.
- Coordinate podcast generation workflows.
- Forward requests to AI processing services.
- Communicate with persistence and storage components.

3. Core Intelligence & Processing Layer (AI Pipeline)

This layer represents the intelligence core of Doc2Pod. It is responsible for transforming raw document content into a structured, context-aware podcast script and converting it into audio.

- **Multi-Modal Extraction:**

  The first stage of the processing pipeline. It is responsible for extracting and interpreting content from PDF documents using advanced OCR and vision-language models, including **PaddleOCR-VL** and **Qwen3-VL**. It supports both digitally generated and scanned PDFs while preserving the semantic meaning and structure.

- **Semantic Chunking & Vector Embedding:**

  Responsible for dividing document content into semantically meaningful chunks and converting them into vector embeddings . These embeddings preserve the contextual meaning of the content and support multilingual processing, enabling efficient semantic search and retrieval. The generated document embeddings are stored in the Vector Database and later used to provide relevant context during the podcast generation process.

- **Retrieval-Augmented Generation (RAG) System:**

  Retrieves the most relevant information from the indexed document content using semantic similarity search. By grounding the generated content in the source document, it improves factual accuracy and reduces hallucinations.

- **Dialect Adapter (Qwen3):**

  Generates a conversational podcast script from the retrieved document content using the **Qwen3** model. The generated script is adapted to the **Egyptian Arabic dialect**, transforming formal document information into natural, engaging spoken language.

- **Speech Engine (Text-to-Speech)**:

  The final stage converts the generated script into natural-sounding audio using the **VibeVoice** text-to-speech model. It produces clear and natural Egyptian Arabic speech, ensuring an engaging listening experience. The generated audio is then saved as downloadable MP3 podcast files for easy access and playback.

4. Data Persistence Layer

The Data Persistence Layer stores all system data required for operation, monitoring, and retrieval.

- **Relational Database (SQL Server):** Used to store and manage structured application data, including user information, podcast metadata, processing records, and system logs.
- **Vector Database (ChromaDB):** Used to store document embeddings and chunked text segments, enabling efficient semantic search and retrieval within the RAG pipeline.
- **Cloud Storage (Supabase Storage):** Responsible for storing original PDF documents and generated MP3 podcast files, providing scalable and efficient storage for large files.

3.1.2 System Users

A. Intended Users

Doc2Pod is designed for individuals and organizations that need to convert document content into audio podcasts in Egyptian Arabic. The system is intended for users who prefer consuming information through natural and engaging spoken content rather than reading lengthy documents. The intended users include:

- **Computer Science Students:** To convert study materials, lecture notes, and academic documents into Egyptian Arabic audio podcasts for easier learning and revision.
- **Computer Science Researchers and Academics:** To review research papers, journal articles, and technical reports in a more accessible audio format.

B. User Characteristics

The system is designed to be user-friendly and does not require advanced technical skills. Users are expected to have:

- Basic computer and web browsing skills.
- The ability to upload PDF documents and navigate web applications.
- Basic familiarity with audio playback controls.
- No prior knowledge of artificial intelligence, machine learning, or document processing technologies is required.

3.2 Design Diagrams

3.2.1 Use Case Diagram

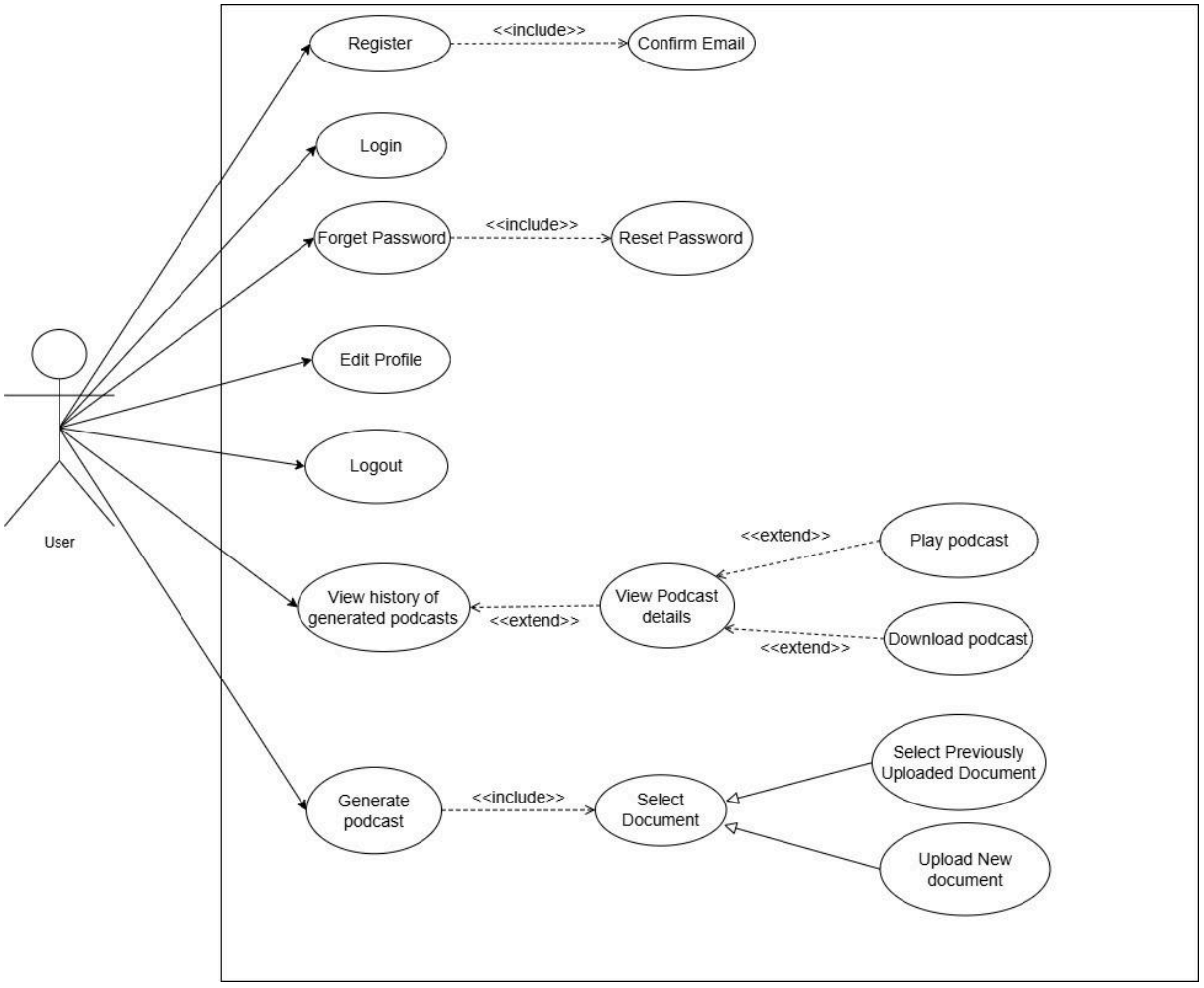

Figure 3.2.1.1 Use Case Diagram

Use Case Description

Actor: User (Normal User)

Users are individuals who use the Doc2Pod application to extract text from documents and convert them into audio podcasts. Their functionalities include:

- **Register (Includes Confirm Email):**

  o **Description**: Allows new users to create an account within the system. The registration process strictly requires email confirmation to verify the user's identity and activate the account.

  o **Pre-conditions**: The user must not have an existing active account with the provided email.

  o **Post-conditions**: A temporary inactive user account is created, and a verification link is dispatched to the user's email for activation.

- **Login:**

  o **Description**: Enables registered users to authenticate and securely access the application by entering their credentials (Email and Password).

  o **Pre-conditions**: The user must have a registered and email-confirmed account.

  o **Post-conditions**: The user is securely authenticated and granted access to the application dashboard.

- **Forget Password (Includes Reset Password):**

  o **Description:** Provides a recovery mechanism for users who have lost their credentials. Initiating this request automatically includes the process of resetting the password via a secure link.

  o **Pre-conditions:** The user must provide a valid email registered within the system.

  o **Post-conditions:** A secure password reset link is sent to the user, allowing them to update their password credentials.

- **Edit Profile:**

  o **Description:** Permits users to update their personal profile details, such as their name, email, password, or contact information.

  o **Pre-conditions:** The user must be successfully logged into the system.

  o **Post-conditions:** The user's profile information is updated in the database, and changes are reflected instantly across the interface.

- **Logout:**

  o **Description:** Securely terminates the user's active session and logs them out of the application.

  o **Pre-conditions:** The user must be currently logged into an active session.

  o **Post-conditions:** The current session token is invalidated, and the user is redirected back to the landing or login page.

- **Generate Podcast (Includes Select Document):**

  o **Description:** The core functionality of the system, enabling users to initiate the AI pipeline to transform a document into an audio podcast. This mandatory process includes the step of selecting a document source.

  o **Pre-conditions:** The user must be logged in and have an active, authorized session.

  o **Post-conditions:** A natural-sounding audio podcast is successfully generated, indexed in the user's history, and stored ready for playback or download.

- **Select Document (Generalized into Upload New Document / Select Previously Uploaded Document):**

  o **Description:** A base functionality that handles document selection for podcast generation. Since this is a generalized use case, the user must perform it through one of two specific methods:

    - Upload New Document: Allows the user to browse and upload a new PDF file from their local device to the system.
    - Select Previously Uploaded Document: Enables the user to choose a PDF file that they have already uploaded and indexed in their document history.

  o **Pre-conditions:** The user must trigger the "Generate Podcast" flow.

  o **Post-conditions:** The chosen document is successfully validated by the system and designated as the text source for the upcoming podcast generation.

- **View History of Generated Podcasts (Extended by View Podcast Details):**

  o **Description:** Displays a list of all successfully generated and completed podcasts created by the user. The user can browse previously generated podcasts and select any podcast to view its detailed information, audio file, generation date, and related metadata.

  o **Pre-condition:** The user must navigate to the Library Dashboard.

  o **Post-condition:** The system retrieves and displays all completed podcasts associated with the user account. Upon selecting a podcast, the system presents its detailed view.

- **View Podcast Details (Extended by Play Podcast / Download Podcast):**

  o **Description:** Shows a dedicated screen containing the metadata and specific information of the selected podcast. From this detailed view, the functionality can be optionally extended based on the user's interaction:

    - Play Podcast: Allows the user to stream and listen to the generated audio directly within the web browser.
    - Download Podcast: Allows the user to download the final generated audio as an MP3 file to their local device.

  o **Pre-conditions:** The user must select a specific podcast from their history list.

  o **Post-conditions:** The system renders the dedicated player interface containing the detailed metadata, inline audio controls, and download triggers.

3.2.2 Class Diagram

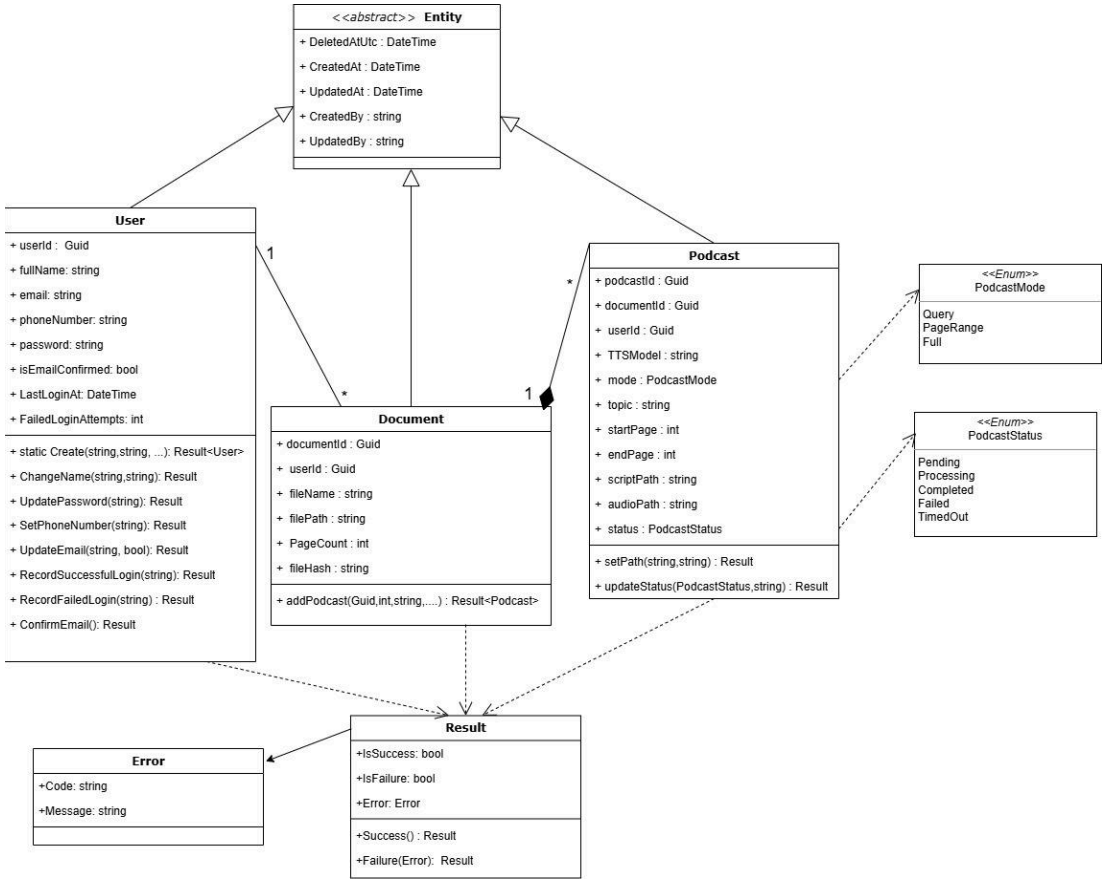

Figure 3.2.2.1 Class Diagram

Class Description

Entity (Abstract Class)

Description: The Entity class serves as the abstract base class for the system's domain entities. It provides common auditing attributes such as creation, modification, and deletion information, ensuring consistency and reducing duplication across the domain model.

Main Attributes:
CreatedAt, UpdatedAt, DeletedAtUtc, CreatedBy, and UpdatedBy.

User Class

Description: The User class represents a registered user of the system. It stores personal information, authentication-related data, and account activity records required for managing user access and interactions with the application.

Main Attributes: Includes user details such as full name, email address, phone number, password, email confirmation status, last login date, and failed login attempts.

Main Responsibilities: Supports user account management operations including account creation, profile updates, password management, email confirmation, and login activity tracking.

Document Class

Description: The Document class represents a document uploaded by a user and serves as the primary source for podcast generation. It stores document metadata and manages the podcasts generated from its content.

Main Attributes: Includes document information such as file name, file path, page count, file hash, and the identifier of the document owner.

Main Responsibilities: Supports document management operations and podcast creation by generating and maintaining podcasts associated with the uploaded document.

Podcast Class

Description: The Podcast class represents a podcast generated from a specific document. It stores podcast configuration settings, generated content references, processing information, and output file locations throughout the podcast generation lifecycle.

Main Attributes: Includes podcast mode, topic, page range, script path, audio path, processing status, and references to the associated user and source document.

Main Responsibilities: Supports podcast generation tracking, output file management, and status updates during the processing lifecycle.

PodcastMode Enumeration

Description: Defines the available podcast generation modes that determine how document content is selected and processed for podcast creation.

Values:

- Query
- PageRange
- Full

PodcastStatus Enumeration

Description: Represents the different processing states of a podcast throughout its generation lifecycle, allowing the system to monitor progress and handle processing outcomes.

Values:

- Pending
- Processing
- Completed
- Failed
- TimedOut

Supporting Classes

Description: Provides a standardized mechanism for operation result handling and error reporting within the system.

Classes:

- **Result:** Represents the success or failure of an operation.
- **Error:** Represents error details associated with failed operations.

3.2.3 Sequence Diagram

A) Podcast Generation Flow

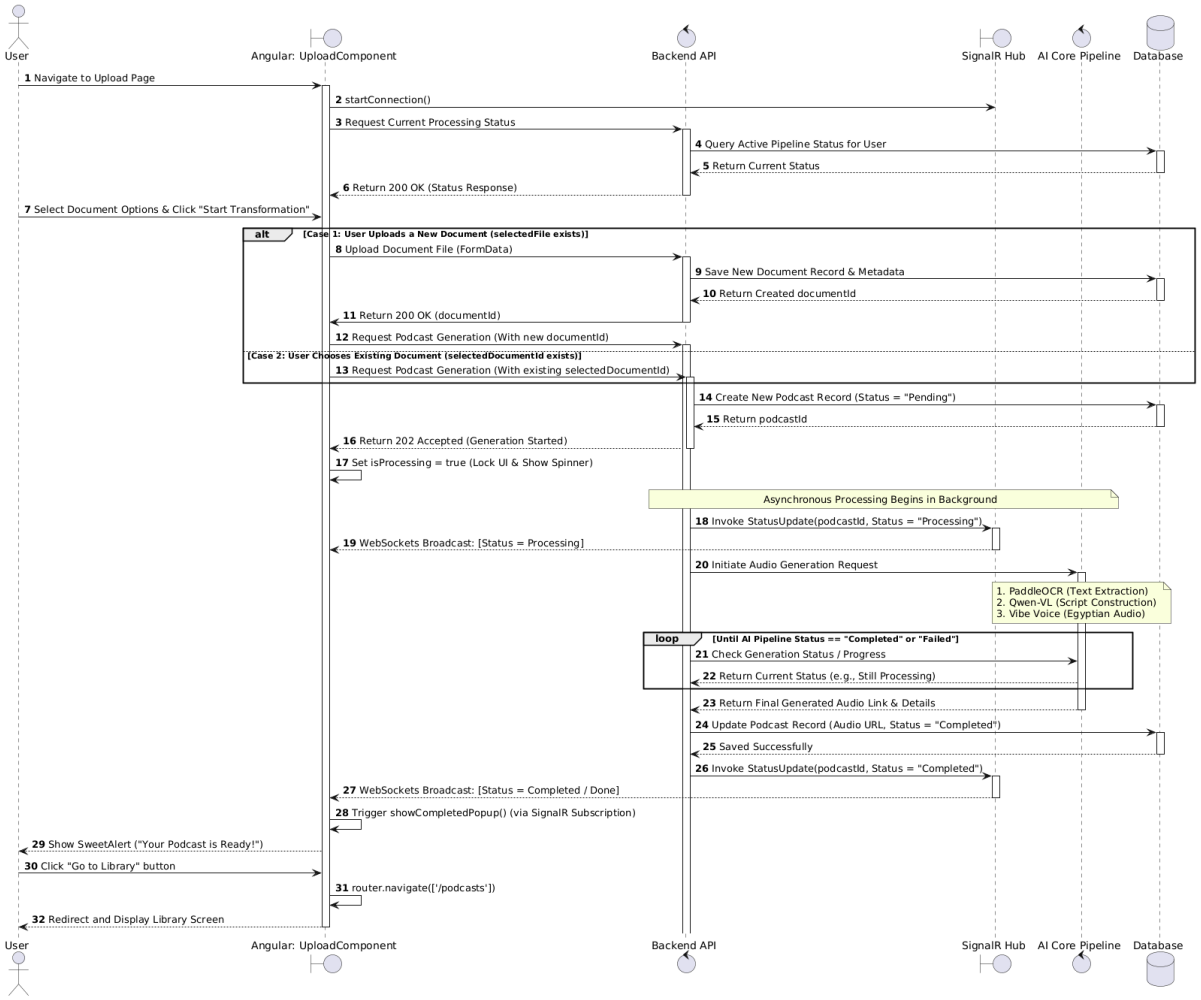

Figure 3.2.3.1 Sequence Diagram: Podcast Generation Flow

B) Podcast Details and Playback

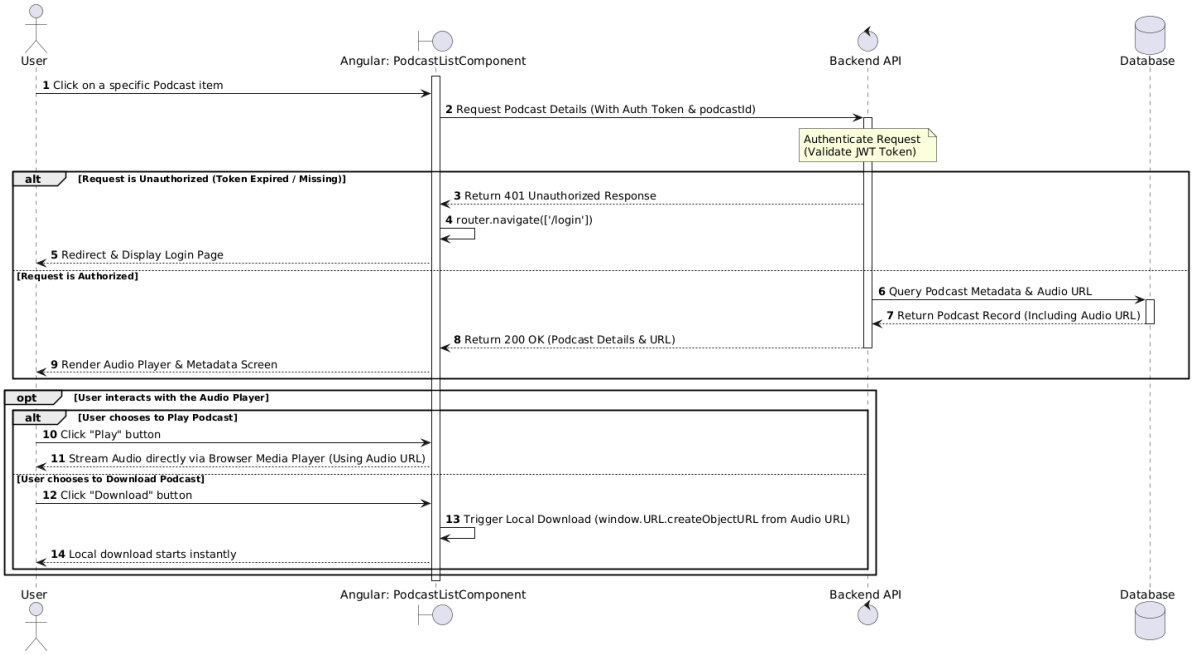

Figure 3.2.3.2 Sequence Diagram: Podcast Details and Playback

3.2.4 Database Diagram

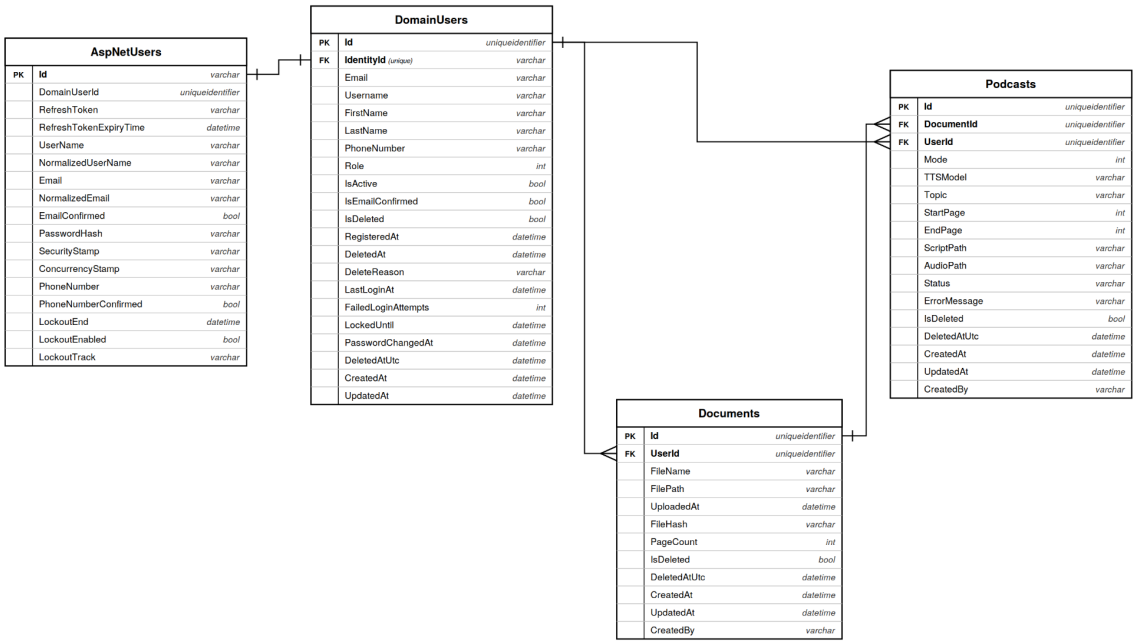

Figure 3.2.4.1 Database diagram

Database Schema Specification

AspNetUsers Table

Description: Manages user authentication and identity information using the ASP.NET Core Identity framework. This table is responsible for handling login credentials, security-related data, and access control mechanisms required for user authentication.

Main Data: Stores account credentials, authentication data, security settings, password management information, and token-related information, including refresh tokens and their expiration settings.

Relationships: Establishes a one-to-one relationship with the DomainUsers table.

DomainUsers Table

Description: Represents the core application user profile, separating business-related user information from the authentication layer. This design improves maintainability by isolating application logic from identity management concerns.

Main Data: Stores user profile information, role assignments, account status indicators, activity records, and auditing data required by the application's business processes.

Relationships: Establishes a one-to-one relationship with the AspNetUsers table and one-to-many relationships with both the Documents and Podcasts tables.

Documents Table

Description: Stores documents uploaded by users and serves as the primary input source for the podcast generation process. It supports document management and tracking throughout the processing lifecycle.

Main Data: Stores document metadata, file details, ownership information, upload records, and auditing data associated with document processing and management.

Relationships: Establishes a many-to-one relationship with the DomainUsers table and a one-to-many relationship with the Podcasts table.

Podcasts Table

Description: Stores generated podcast records and information related to the podcast creation process. Each podcast is generated from a specific document and associated with a particular user.

Main Data: Stores podcast generation settings, generated content references, processing status information, error records, and auditing data required throughout the podcast lifecycle.

Relationships: Establishes many-to-one relationships with both the DomainUsers and Documents tables.

4- Implementation and Testing

4.1 Dataset Description

The architectural complexity of the Doc2Pod system necessitates a highly specialized, dual-domain data ecosystem. Because the framework bridges multimodal academic document understanding with dialect-specific conversational speech synthesis, off-the-shelf public datasets were fundamentally incompatible with the project's objectives. Consequently, the system relies on two distinct foundational datasets: an expansive, custom-engineered **Acoustic Code-Switched Speech Corpus** for fine-tuning the Text-to-Speech (TTS) engine, and a **Complex Academic Knowledge Base** that serves as the contextual grounding for the Retrieval-Augmented Generation (RAG) pipeline.

The following subsections provide an exhaustive breakdown of the characteristics, statistical volumes, and structural properties of these datasets.

4.1.1 The Acoustic Code-Switched Speech Corpus

The most expansive and resource-intensive dataset within the Doc2Pod framework is the custom acoustic corpus. This dataset was exclusively designed to solve the challenge of "Code-Switching"—the linguistic phenomenon where a speaker fluently alternates between two languages.

A. Inadequacy of Public Data Domains

Prior to utilizing this custom dataset, extensive evaluations were conducted on existing open-source data repositories. The acoustic corpus used in Doc2Pod explicitly avoids the characteristics of these generic domains due to the following critical data failures:

- **YouTube Scraped Datasets:**
  Publicly scraped audio data is heavily corrupted by background noise, environmental reverb, music, and overlapping multi-speaker interference. In the context of advanced neural audio synthesis, these acoustic impurities are mathematically catastrophic, as they corrupt the target generation of the model's Diffusion Head.

- **Standard ASR (Automatic Speech Recognition) Datasets:** Standardized Arabic audio datasets suffer from severe speaker inconsistency. Furthermore, they are predominantly recorded in a rigid, monotonous "Reading Style" prosody. This acoustic profile is entirely unfit for Doc2Pod, which strictly requires dynamic, engaging, and expressive conversational podcast intonations.

B. Linguistic and Acoustic Profile

The final dataset is characterized by a precise linguistic mapping. The conversational connective tissue, pacing, and grammatical structures are grounded strictly in authentic Cairene (Egyptian) Arabic colloquialisms. Conversely, all domain-specific Computer Science entities (e.g., *Data Structures*, *Overfitting*, *Binary Trees*) remain strictly in academic English.

Acoustically, the dataset is defined by the following physical properties:

- **Audio Format:**

  Uncompressed, studio-quality Linear Pulse Code Modulation (PCM) audio.

- **Temporal Distribution:**

  To optimize the attention span of the TTS diffusion model and prevent Out-Of-Memory (**OOM**) tensor overflow during training, every single audio segment in the dataset is strictly bounded between **5.0 and 9.0 seconds** in duration.

C. Volumetric Scale and Database Distribution

The raw scale of the dataset represents a massive collection of individual utterances, partitioned across multiple data nodes (representing the collective efforts of the engineering team). The aggregate raw database contains **101,724 isolated audio segments**. The exact structural distribution of the original raw data chunks is as follows:

- **Node 1 Data Array:** 20,594 rows
- **Node 2 Data Array:** 29,704 rows
- **Node 3 Data Array:** 10,067 rows
- **Node 4 Data Array:** 23,249 rows
- **Node 5 Data Array:** 8,046 rows
- **Node 6 Data Array:** 10,064 rows

D. Data Integrity and The Final Benchmark Corpus

Following rigorous cross-referencing and validation against character corruption, the dataset achieved a perfect integrity status. The validation matrix confirmed that **no "ghost files"** (audio files lacking textual counterparts) were present, and all matched transcripts retained **100% character identity** without structural corruption.

From this massive pool of **101,724 segments**, the data was strictly filtered to produce the definitive **Doc2Pod Training Benchmark**. This finalized subset consists of **30,000 clean, high-fidelity Egyptian conversational records**. In total, this curated acoustic dataset represents approximately **120 hours of continuous, perfectly aligned studio-quality voice data**, providing the robust foundation necessary for localized acoustic modeling.

4.1.2 The Academic Knowledge Base

While the TTS dataset consists of tightly controlled, perfectly aligned audio-text pairs, the input dataset used for evaluating the RAG retrieval system is inherently unstructured, dense, and visually complex.

- **Topical Domain:**

  The dataset comprises collegiate-level Computer Science materials, encompassing subject matter such as software engineering primitives, complex algorithm mechanics, system architecture paradigms, and networking protocols.

- **Structural Characteristics:**

  Unlike standard raw text corpora (such as plain text files or clean JSON arrays), this dataset is ingested directly in its native academic formats—primarily Portable Document Format (PDF) files.

- **Data Complexity:**

  The documents in this corpus are characterized by extreme geometric and visual complexity. The data includes non-linear reading flows (such as multi-column textbook layouts), deeply nested hierarchical bullet points, structural comparison tables, and embedded technical syntax (including multi-line programming code blocks and mathematical derivations). These characteristics define a dataset that is highly dense and requires significant spatial

awareness to interpret correctly without fracturing the semantic meaning of the educational content.

4.1.3 The Egyptian Podcast Instruction Dataset

One of the main contributions of this project is the creation of a high-quality, domain-specific instruction dataset for fine-tuning large language models on Egyptian Arabic conversational content, this corpus was specifically designed to train the model to generate natural, engaging, and technically accurate podcast-style dialogues in Cairene Egyptian Arabic.

A. Motivation and Limitations of Existing Datasets

Public Arabic dialogue datasets are generally insufficient for this project's objectives. Most available corpora are either written in formal Modern Standard Arabic (MSA), lack technical depth, or fail to reflect authentic Egyptian conversational style. They rarely combine deep Computer Science concepts with natural code-switching (mixing Egyptian colloquial Arabic with English technical terms) in a podcast format. Therefore, a custom dataset was necessary to achieve the desired output quality.

Real-world technical documents (PDFs) used in the Doc2Pod system are often very long and cannot be fed to the language model in one pass due to context length limitations. Therefore, documents are processed in smaller chunks (sections and sub-parts).

To ensure the model can handle this chunked workflow effectively, we designed the training data to simulate real usage scenarios. The model needed to learn how to:

- Generate a podcast segment for **any part** of a document.
- Maintain continuity whether the segment is the **beginning**, **middle**, or **continuation** of a topic.
- Produce smooth transitions between sections.
- Avoid losing context or hallucinating when dealing with partial information.

B. Dataset Generation Pipeline

The dataset contains approximately **5,000 high-quality conversation samples**. It was generated using **Google Gemini** through an automated pipeline . For each technical topic, the model first produced structured

technical content divided into two parts . These parts were then converted into full podcast-style dialogues between **Sara** (the host) and **Ahmed** (the senior engineer) using highly detailed prompting.

Each training sample includes rich contextual information such as:

- Current topic and section position.
- Document structure.
- Previous summary.
- Specific continuity and transition rules.

This design forces the model to learn how to speak naturally in Egyptian Arabic regardless of whether it is starting a new topic, continuing from previous content, or wrapping up a section.

4.2 Phases Description

This section provides a rigorous, algorithmic breakdown of the methodologies, system states, and pipelines implemented across the core execution phases of the project.

4.2.1 Phase 1: Automated Data Collection

To train a highly specialized, localized text-to-speech model, a massive volume of perfectly aligned audio-text pairs was required. Since manual recording was unfeasible, a fully automated data generation and temporal segmentation microservice was engineered (managed via **producer** and **Segmenting**). This pipeline was designed to handle API integration, error recovery, linguistic formatting, and precise audio slicing.

A. Prompt Engineering for Bilingual Code-Switching

The first stage of the pipeline focused on generating the base conversational scripts. A Large Language Model (LLM) was utilized as the generation engine. To produce authentic Egyptian Arabic podcast scripts that accurately discussed Computer Science topics, strict prompt engineering constraints were applied:

- **Contextual Seeding:** The pipeline systematically iterated through a predefined list of computer science domains (e.g., Data Structures, DevOps, System Design).

- **Linguistic Rule Enforcement:** The prompt dynamically instructed the LLM to write the narrative connective tissue in Cairene Egyptian Arabic while forcing all core technical terminology to remain in standard English characters. This controlled generation ensured the resulting data would teach the downstream acoustic model how to correctly "code-switch" between phonologies without translating technical entities.

B. Fault-Tolerant API Rotation System

Generating thousands of audio files programmatically introduces severe network and rate-limiting bottlenecks. To ensure continuous operation without manual intervention, a robust fault-tolerance mechanism was implemented:

- **Key Management:**

  The system initialized a pool of cryptographic API keys managed by a cyclic iterator.

- **Exception Interception:**

  During generation, if the host server returned an HTTP **429 Too Many Requests** error (indicating quota exhaustion), the pipeline's exception handler instantly caught the failure.

- **State Recovery:**

  Instead of crashing, the state machine automatically deactivated the exhausted key, rotated to the next available key in the pool, and re-attempted the exact text-to-audio pair. This ensured zero data loss and uninterrupted pipeline execution.

C. Audio Standardization and Synchronization

As the generated audio streams were received, they required immediate standardization to fit the strict input dimensions of the target TTS diffusion models.

- The raw audio payloads were intercepted and uniformly converted into uncompressed .wav formats with a fixed sample rate of **24,000 Hz** (Mono, 16-bit).

- To maintain database integrity, an automated file-mapping function was written. Every saved audio file was dynamically named based on its exact duration and topic identifier, and its corresponding text transcript was saved with an identical matching name. This guaranteed a perfect 1:1 synchronization between text and audio across the distributed database.

D. Word-Level Forced Alignment and Segmentation

Because the generated audio clips often contained multiple sentences, feeding them directly into an acoustic model would cause severe memory overflow during fine-tuning. The audio needed to be sliced into smaller chunks (ideally between 4 to 9 seconds). However, arbitrary mathematical slicing risks cutting words in half, destroying the phonetic data.

- To solve this, the pipeline integrated an Automatic Speech Recognition (ASR) model to perform **Forced Alignment**.
- The alignment algorithm mapped the text transcript directly to the audio waveform, generating precise millisecond timestamps for the start and end of every single word.
- Utilizing these temporal markers, a custom Python segmenting script safely spliced the audio only during natural pauses and spaces between words, resulting in perfectly clean, isolated contextual clips.

E. Automated Signal Cleaning (Silence Suppression)

As a final preprocessing step, the isolated audio clips were passed through an automated cleanup filter.

- **Peak Normalization:** Files were normalized to a consistent volume level to prevent sudden amplitude spikes during model training.
- **RMS Silence Removal:** The script analyzed the Root-Mean-Square (RMS) energy of the audio track. Any prolonged periods of dead silence at the beginning or end of the clips were automatically detected and stripped away. This maximized the density of the training data, ensuring the model only learned from active human speech patterns.

4.2.2 Phase 2: Intelligent Document Extraction

The secondary phase of the system architecture serves as the ingestion and comprehension engine. Its primary objective is to convert visually complex, unstructured academic documents (e.g., PDFs) into a clean, machine-readable, and geometrically ordered format suitable for vectorization.

A. The Challenge of Unstructured Multimodal Data

Academic documents inherently lack linear machine readability. They utilize complex spatial geometries to convey meaning, heavily featuring multi-column text flows, embedded mathematical equations, structured tables, and deep-context diagrams. Standard text-extraction algorithms operate on a basic horizontal-sweeping paradigm (reading strictly left-to-right, top-to-bottom). When applied to academic PDFs, this linear approach catastrophically scrambles multi-column layouts, merges unrelated paragraphs, and completely destroys the syntactic structure of mathematical formulas and code blocks.

B. Algorithmic Evolution and Multi-Model Integration

To solve the layout parsing dilemma, the extraction pipeline underwent a rigorous iterative engineering process, ultimately resulting in a hybrid multi-model architecture:

1. **Iteration 1 (Standard Optical Character Recognition):**

   Initial extraction attempts utilized standard OCR models. While highly accurate for flat, single-column prose, this approach failed to recognize spatial relationships. Tables were flattened into incoherent text strings, and equations were corrupted.

2. **Iteration 2 (Layout-Aware Vision-Language Parsing):**

   The architecture was upgraded to utilize a layout-aware Vision-Language (VL) model. This model successfully mapped document structures, cleanly identifying bounding boxes for tables, code, and text blocks. However, a critical limitation was discovered: while it recognized that a diagram or chart existed, it treated it merely as a visual spatial element and could not extract

the underlying scientific insights or trends depicted within the image.

3. **Iteration 3 (The Hybrid Integration Architecture):**

   Recognizing that a single model was insufficient for total comprehension, a dual-agent multimodal pipeline was engineered. The system integrates the structural precision of a Layout-Aware VL model with the deep reasoning capabilities of a secondary Multimodal Large Vision Model (LVM).

C. The Hybrid Extraction Routing Pipeline

The finalized extraction microservice operates through a dynamic routing matrix:

- **Spatial Decomposition:**

  As a document page is ingested, it is rendered as an image tensor. The Layout-Aware model performs spatial decomposition, drawing categorical bounding boxes around discrete elements (e.g., Title, Text, Table, Equation, Figure).

- **Element Routing:**

  * Identified Text, Table, and Equation coordinates are processed to maintain their internal formatting (e.g., converting tables into converting tables into readable structured text).

    - Identified Figure and Chart coordinates are dynamically cropped and routed to the deep Multimodal LVM. This model analyzes the pixels of the chart and generates a dense textual summary of the visual data (e.g., explaining the trend line of a graph).

- **Semantic Reassembly:** The pipeline recombines the extracted text, structured tables, and generated image summaries into a single, cohesive linear data stream that perfectly matches the original human reading order.

D. Adaptive Semantic Chunking

Once the raw document is transformed into a continuous, structurally sound text stream, it must be partitioned. Feeding an entire textbook

chapter into an LLM simultaneously exceeds context window limitations and dilutes attention.

The pipeline applies an **Adaptive Semantic Chunking** algorithm. Rather than slicing the text using arbitrary character limits (which could fracture sentences or separate a table from its explanation), the algorithm tokenizes the text based on semantic structural boundaries (such as heading shifts).

4.2.3 Phase 3: Advanced Retrieval-Augmented Generation (RAG)

With the raw document parsed into a continuous text stream, the system requires an advanced retrieval engine to fetch precise technical data for the script generation phase. This phase implements a hybrid search architecture designed specifically to eliminate temporal and structural hallucinations.

A. Context-Aware Concept Grouping and Semantic Fusion

Before text chunks are embedded into the dense vector database, they undergo an Intelligent Semantic Fusion process to ensure no educational concepts are left fragmented.

- **Algorithmic Fusion:** The system calculates the dense vector overlap between sequential layout concepts. If two sequential headings share a Cosine Similarity of $\geq 94\%$, or if a Regular Expression (RegEx) engine detects continuation patterns, the algorithm programmatically fuses them into a single, unified semantic block.
- **Page-Aligned Sub-Chunking:** To guarantee seamless integration with the subsequent Large Language Model (LLM) context window without causing attention dilution, these concepts undergo a hard-boundary limitation check (maximum length of 1800 characters). To avoid arbitrary slicing, oversized chunks are partitioned strictly along their original PDF page boundaries, optimizing the text for dense Computer Science theoretical components.

B. Resolving "Cross-Chapter Hallucination"

One of the most critical failures in standard RAG pipelines is "structural blindness." A pure vector search relies exclusively on mathematical vocabulary overlap. Consequently, if a user queries a basic foundational definition from Chapter 1, a standard similarity search might confidently retrieve a highly complex, advanced algorithm from Chapter 10 simply

due to matching keywords. This causes the generation model to hallucinate content completely out of chronological educational order.

C. Multi-Mode Retrieval Routing Architecture

- **Iteration 1 (Dual mode Retrieval Routing Architecture)**
  To explicitly solve cross-chapter hallucinations, a dual-mode deterministic retrieval system was engineered:

  1. **Mode 1 (Auto-Overview & Smart Skeleton Implementation):**

     When the system needs to generate the global introduction or "teaser" for the podcast episode, standard retrieval fails because it lacks full-document awareness. To fix this, a dynamic layout parser was implemented. It iteratively scans the entire structured document hierarchy, extracts exactly the first **300 characters** of every unique concept, and compiles them into a global "Rich Skeleton." This overarching roadmap is injected directly into the system prompt, guaranteeing the AI can accurately forecast upcoming topics without hallucinating non-existent material.

  2. **Mode 2 (Hybrid Bounded Search):**

     For the core educational generation, the retrieval engine fuses dense vector cosine similarity with strict, deterministic metadata filtering. By explicitly clamping the vector search space using specific page-range metadata parameters, the algorithm physically locks the retrieval engine to the chronological progression of the academic document.

- **Iteration 2 (3-Mode Retrieval Routing)**
  To explicitly solve cross-chapter hallucinations and ensure chronological coherence, the retrieval architecture underwent a critical evolution. Initially, the system relied on a heuristic "Rich Skeleton" approach (extracting the first 300 characters of every concept to forecast topics). However, empirical testing revealed that heuristic summarization degrades technical fidelity in complex Computer Science proofs and code blocks.Consequently, the architecture was refactored into a Deterministic, Metadata-Aware

3-Mode Routing System powered by ChromaDB. This engine dynamically routes retrieval queries based on the user's intent, fusing dense vector cosine similarity with strict spatial constraints:

1. **Mode 1 Unbounded Semantic Retrieval (Topic-Focused) :**

   When the user queries a specific concept, the system encodes the query using the BGE-M3 embedding model and performs an unconstrained vector search across the entire document corpus. This mode isolates the exact subchunks containing the technical definition regardless of their physical location in the document, optimized for targeted concept clarification with high semantic relevance and no spatial bias.

2. **Mode 2 Hybrid Bounded Search (Spatially Constrained):**

   For the core educational generation, the retrieval engine fuses dense vector cosine similarity with strict, deterministic metadata filtering. By explicitly clamping the vector search space using specific page-range metadata parameters, the algorithm physically locks the retrieval engine to the chronological progression of the academic document.

3. **Mode 3 Sequential Full-Document Streaming (Auto-Overview Evolution)**

   For the core educational generation, the retrieval engine fuses dense vector cosine similarity with strict, deterministic metadata filtering. By explicitly clamping the vector search space using specific page-range metadata parameters, the algorithm physically locks the retrieval engine to the chronological progression of the academic document.

4.2.4 Phase 4: Language Model Adaptation & Dialectal Script Synthesis

The cognitive core of Doc2Pod requires translating highly complex, academic English into a fluent, conversational Egyptian Arabic dialect while maintaining strict technical accuracy. Because base foundational LLMs inherently lack native fluency in specific regional colloquialisms, an advanced model adaptation pipeline was required.

A. Resource-Constrained Dialect Transfer (QLoRA Pipeline)

Full-parameter fine-tuning of a multi-billion parameter LLM requires prohibitive amounts of VRAM, compute clusters, and time. To bypass these hardware constraints while injecting native Egyptian linguistic capabilities, an efficient Parameter-Efficient Fine-Tuning (PEFT) pipeline was constructed:

- **Weight Quantization:**

  The base foundational model is loaded into memory using 4-bit Quantization. This mathematically compresses the precision of the model's floating-point weights, drastically reducing the active memory footprint during the backward optimization pass.

- **Low-Rank Adaptation (LoRA):**

  Instead of altering the frozen pre-trained base weights, the pipeline injects trainable low-rank decomposition matrices (configured with a Rank = 32 into the attention layers. This allows the model to learn the specific morphological rules of the Egyptian dialect efficiently.

B. Optimization via Instruction Masking

During standard fine-tuning, the loss function penalizes the model based on the entire prompt sequence. However, in Doc2Pod, the system prompts and retrieved contexts are written in academic English, while only the desired target output is in Egyptian Arabic.

To prevent the optimizer from corrupting the model's English technical comprehension, **Response-Only Tuning (Instruction Masking)** was implemented. The loss calculation is mathematically masked across the user instruction and RAG context tokens. Gradient updates are strictly calculated and applied *only* to the Assistant's generated dialogue tokens, forcing the model to hyper-focus purely on its conversational output persona.

C. State-Aware Script Generation

Because full podcast episodes far exceed standard LLM maximum context windows, the generation pipeline utilizes a State-Aware Generation loop. Rather than processing the podcast as a single massive prompt, the script is generated sequentially in isolated segments.

To preserve the continuity of the multi-speaker persona (the "Host" and the "Expert") across these independent segments, the system employs **Accumulated Summarizing**. A secondary background agent condenses the core conversational state from previous chunks and prepends it to the active memory of the current generation step. This creates an illusion of infinite memory, allowing the podcast to maintain continuous logical references throughout a 30-minute episode.

In addition, each generation step receives a structured representation of the source document, including its **hierarchical organization**, section boundaries, and metadata. This document structure provides global awareness of the material being discussed and helps the agents maintain consistency with the original content.

Furthermore, the system uses **dynamic prompt templates** that adapt according to the position of the current segment within the podcast. Early segments focus on introducing the topic and establishing context, middle segments emphasize detailed explanations and transitions between sections, while final segments prioritize summarization and concluding remarks. By combining accumulated memory, document structure awareness, and position-aware prompting, the system generates long-form conversations that remain coherent, contextually grounded, and naturally progressive.

4.2.5 Phase 5: Acoustic Modeling & TTS Fine-Tuning

The speech synthesis module represented the most technically demanding component of the project, requiring the adaptation of large-scale diffusion-based text-to-speech (TTS) models to reproduce the phonetic and prosodic characteristics of code-switched Egyptian Arabic. Rather than committing to a single architecture from the outset, the implementation investigated two state-of-the-art frameworks in parallel: **VibeVoice 1.5B** and **F5-TTS**. Both models were evaluated with respect

to computational efficiency, fine-tuning stability, speech naturalness, and their ability to preserve the intended conversational "Doc2Pod" speaking style.

Although F5-TTS offered several architectural advantages for future development, it has not yet achieved production-ready results under the project's available computational constraints. Consequently, the current implementation adopts a hybrid synthesis architecture. To guarantee studio-quality audio for the end-user, the system utilizes a high-fidelity external TTS API (gemini-2.5-flash-preview-tts) as the primary generation tier. However, the architecture continues to rely on our successfully fine-tuned VibeVoice model as the core local fallback engine, as it remains the only offline framework to have successfully produced intelligible and usable code-switched speech during our training conditions.

A. Acoustic Adaptation (VibeVoice 1.5B Framework)

VibeVoice 1.5B served as one of the primary candidate architectures due to its diffusion-based speech generation pipeline and its ability to produce high-quality synthesized speech while remaining compatible with parameter-efficient adaptation techniques. Its implementation primarily focused on overcoming hardware limitations during model adaptation.

- **Compute Bottleneck & Parameter-Efficient Tuning:**
  Full fine-tuning of a 1.5-billion-parameter model exceeds the computational capacity typically available in undergraduate research environments. To enable dialect adaptation, the training pipeline employed Low-Rank Adaptation (LoRA), introducing trainable low-rank matrices into the attention layers while freezing the original model parameters. This reduced the number of trainable parameters to less than 1% of the complete model, making fine-tuning computationally feasible.

- **Memory Optimization via Weight Quantization:**
  Training a model of this scale on GPUs with approximately 16 GB of VRAM frequently resulted in Out-of-Memory (OOM) errors. To reduce memory consumption, the model was loaded using 8-bit weight quantization, significantly decreasing VRAM requirements while preserving the linguistic knowledge contained in the pre-trained model.

- **Training Continuity under Cloud Constraints:**
  Since cloud GPU sessions may terminate unexpectedly before completing long training runs, the training corpus was partitioned into serialized data shards that could be processed independently. Combined with periodic checkpointing, this allowed interrupted sessions to resume from the most recently completed shard instead of restarting the entire training process.

B. Parallel Evaluation of the F5-TTS Framework

In parallel with the VibeVoice implementation, the project also investigated the F5-TTS architecture because of its flow-matching formulation, simplified acoustic modeling pipeline, and reported improvements in prosody modeling and training stability. These characteristics made F5-TTS an attractive long-term candidate for dialect adaptation.

- **Flow-Matching Fine-Tuning:**
  Unlike VibeVoice's generation pipeline, F5-TTS is built upon conditional flow matching. The fine-tuning process was adapted accordingly, optimizing the model to learn speech trajectories that better represent the rhythm and pronunciation patterns of conversational Egyptian Arabic while maintaining the pronunciation of embedded English technical terminology.
- **Code-Switching Adaptation:**
  Particular attention was given to preserving the pronunciation of mixed-language expressions, preventing English technical terms from being over-Arabized while maintaining natural pronunciation for Arabic words appearing within the same sentence.
- **Checkpoint Synchronization:**
  To improve robustness against interrupted cloud sessions, an automated checkpoint synchronization routine periodically backed up intermediate model states to external cloud storage. Upon restarting a training session, the most recent checkpoint and optimizer state could be restored automatically, minimizing lost computation.

Despite these architectural advantages and considerable experimentation, the F5-TTS implementation has not yet produced synthesized speech of sufficient quality to be considered a successful fine-tuning outcome.

While the framework remains the preferred direction for future research due to its promising design and reported capabilities, additional experimentation with training strategies, larger datasets, and extended computational resources is still required before it can replace the current production pipeline.

C. Dialectal Persona Engineering

Across both candidate architectures, the primary objective was to preserve the conversational "Doc2Pod Persona" rather than generating neutral read-aloud speech. Accordingly, the fine-tuning corpus emphasized instructional dialogue and naturally spoken explanations instead of formal narration.

The training data was curated to expose the models to conversational Egyptian Arabic mixed with English technical terminology, encouraging consistent pronunciation, natural rhythm, and an instructional speaking style appropriate for educational podcast generation.

4.3 Technologies & System Architecture

This section details the comprehensive software stack, frameworks, and architectural design patterns utilized to bring the Doc2Pod platform into production. To ensure seamless communication between user-facing web traffic and intensive deep learning workloads, the system was decoupled into three independent technological tiers: a component-based frontend, a robust backend orchestrator, and an isolated AI microservice.

4.3.1 Frontend Web Architecture

The presentation layer of Doc2Pod was designed as a highly scalable Single Page Application (SPA).

- **Framework & Languages:**

  The application was built using **Angular** combined with **TypeScript**, **HTML5**, and **CSS3**. Angular was selected for its robust component-based architecture, advanced dependency

injection, and optimized routing mechanisms, which prevent full-page reloads and ensure a fluid user experience during long audio-generation polling sessions.

- **UI Component Library:**

  To accelerate development and guarantee a responsive layout across various screen dimensions, **Bootstrap** was integrated. This provided pre-designed, heavily optimized structural grids and UI elements that conform to modern accessibility standards.

- **API Communication & State:**

  The frontend communicates with the backend via secure RESTful HTTP requests. Angular HTTP Interceptors were utilized to manage stateless user sessions, automatically attaching JSON Web Tokens (JWT) to the headers of outgoing requests to validate authorization.

4.3.2 Backend Orchestrator & Business Logic

The core application server manages user states, file transfers, and request validations.

- **Core Framework:**

  The backend was implemented using **ASP.NET Core (C#)**, specifically leveraging the **.NET 9 SDK**.

- **Clean Architecture & CQRS:**

  The server strictly adheres to **Clean Architecture** principles, decoupling the business logic from infrastructure and presentation layers. To further isolate state mutations from data retrieval, the **MediatR** library was integrated to enforce the **CQRS (Command Query Responsibility Segregation)** design pattern. This ensures that the codebase remains highly modular, testable, and maintainable.

- **Validation & External Services:**

  All incoming HTTP payloads undergo strict structural verification via **FluentValidation**, ensuring that corrupted data is rejected before hitting the database or the AI processing queues. Additionally, **MailKit** was implemented to provide a secure SMTP interface, managing automated email verifications and password recovery pipelines.

4.3.3 Artificial Intelligence & Data Engineering Stack

Because deep learning models require specialized tensor mathematics and CUDA optimizations, the AI pipeline was isolated into a dedicated processing microservice.

- **Microservice Gateway:**

  The AI service exposes its endpoints using **FastAPI (Python 3.12)**. FastAPI's asynchronous event loop (async/await) ensures high-throughput, non-blocking communication with the ASP.NET Core backend during massive document processing cycles.

- **Multimodal Document Parsing:**

  To extract text while preserving complex visual geometries, the system integrates **PaddleOCR-VL** for structure-aware boundary detection. The isolated layout elements (e.g., charts, complex equations) are then passed to **Qwen-VL**, a large Vision-Language Model capable of deep multimodal reasoning and contextual summarization.

- **Acoustic Modeling & Hybrid TTS:**

  The final Egyptian Arabic script is synthesized into natural human speech using a Fault-Tolerant Hybrid Architecture. The system routes primary generation to a high-fidelity external speech API (gemini-2.5-flash-preview-tts) for studio-grade output, while maintaining a successfully fine-tuned local VibeVoice diffusion model as a resilient offline fallback engine.

- **Deep Learning Framework:**

  All model inference, tensor allocations, and dialect-transfer fine-tuning were executed utilizing the **PyTorch** machine learning

library, optimizing heavy matrix multiplications across isolated GPU hardware.

4.3.4 Database & Security Infrastructure

Data persistence and user security form the foundational backbone of the web application.

- **Relational Database:**

  The project relies on **Microsoft SQL Server** as its primary relational database. This system securely stores user profiles, historical metadata, file state pointers, and podcast staging links.

- **Object-Relational Mapping (ORM):**

  Database schema creation, queries, and complex table relationships are abstracted and managed directly through **Entity Framework (EF) Core**.

- **Authentication and Role-Based Security:**

  A secure, stateless authentication matrix was implemented using **ASP.NET Core Identity** fused with **JWT (JSON Web Tokens)**. Upon passing email verification via MailKit, users are issued cryptographically signed JWTs. The system enforces strict Role-Based Access Control (RBAC), ensuring that specific API endpoints (such as file execution and podcast retrieval) are exclusively available to properly authenticated and authorized identities, while secure hashing algorithms protect backend credential storage.

4.4 Experimental Results

This section details the empirical evaluation of the Doc2Pod system. To ensure academic rigor, the evaluation framework was partitioned to independently test the performance of the Retrieval-Augmented Generation (RAG) knowledge engine, the linguistic richness of the LLM dialect synthesis, and the acoustic naturalness of the custom Text-to-Speech (TTS) architecture.

4.4.1 Knowledge Engineering and Retrieval Precision

The core foundation of the system's factual accuracy relies on its ability to extract, chunk, and retrieve the correct Computer Science concepts. The system was evaluated on chunking fidelity and retrieval accuracy.

A. Semantic Chunking Evaluation

The Adaptive Semantic Chunking pipeline was evaluated using a Hybrid Evaluation Strategy, combining Quantitative LLM-as-a-Judge mechanisms with Qualitative Human-in-the-Loop benchmarking against the original collegiate lecture materials.

- **Global Context Relevance (89%):**

  The chunking algorithm achieved a highly precise 89% semantic alignment between extracted concepts and generated knowledge chunks.

- **CS-Tailored Processing:**

  Human benchmarking confirmed high structural fidelity, demonstrating the algorithm's specific ability to accurately segment dense theoretical content, tables, and complex technical terminology without semantic loss or administrative noise.

B. Retrieval Precision Metrics (Hit Rate & MRR)

To measure the effectiveness of the retrieval engine and evaluate the mitigation of "Cross-Chapter Hallucinations," the system tracked two primary metrics across a benchmark query dataset:

1. **Hit Rate @ K:** A binary metric measuring whether a relevant factual chunk was successfully retrieved within the **top K** results. **Hit Rate = Successful Queries /Total Queries * 100**

2. **Mean Reciprocal Rank (MRR):** A fractional metric evaluating the sorting precision of the engine by measuring the exact rank

position of the first relevant chunk (ranging from 0 to 1
MRR = 1 /Q *(sum(i=1-Q) 1 / Rank_i)

The evaluation contrasted Mode 1 (Pure Vector Search) against Mode 2 (Hybrid Bounded Search).

Table 4.1: Comparative Analysis of Retrieval Precision Metrics

Evaluation Metric	K=3 (Pure Vector)	K=3 (Hybrid Search)	K=10 (Pure Vector)	K=10 (Hybrid Search)
Hit Rate @ K	78.7%	80.9%	83.0%	85.1%
Mean Reciprocal Rank (MRR)	78.7%	80.9%	83.0%	85.1%

Conclusion on Retrieval:

The optimized Hybrid Search consistently outperformed the pure semantic baseline, peaking at an 85.1% retrieval rate at $K=10$. Crucially, the system's MRR perfectly matched its Hit Rate across all thresholds. This mathematically proves that the implemented embedding architecture possesses absolute sorting precision: when the system successfully retrieves the correct technical concept, it consistently places it at the absolute Rank #1 spot, completely eliminating contextual ambiguity for the LLM.

4.4.2 LLM Synthetic Dataset Iteration and Script Evaluation

To ensure the fine-tuned Qwen3-4B model produces high-quality, technically accurate Egyptian Arabic podcasts, we employed a two-phase experimental methodology: (A) Iterative refinement of the synthetic training dataset, and (B) Evaluation of the final fine-tuned model's generated scripts.

A. Iterative Synthetic Dataset Generation

The quality of the fine-tuned model is strictly bounded by the training data. We generated the synthetic dataset through two iterative cycles:

- **Iteration 1 (Baseline):** Utilized mixed Arabic-English prompts without automated evaluation. While this yielded high dialectal naturalness, the resulting data suffered from repetitive filler words (e.g., overusing "تمام" or "فهمت") and occasional technical drift.

- **Iteration 2 (Structured + QA):** Utilized structured English prompts for logical grounding, coupled with an LLM-as-a-Judge evaluation rubric and an Automated Retry Mechanism. This iteration significantly improved factual accuracy and eliminated repetitive looping.

B. Evaluation of Generated Podcast Scripts

The following metrics represent the final evaluation of the scripts generated by the fine-tuned Qwen3-4B models across 20 complete lecture podcasts. Model A corresponds to Iteration 1 (trained on mixed Arabic-English prompts), while Model B corresponds to Iteration 2 (trained on structured English prompts). The evaluation employed a qualitative LLM-as-a-Judge approach.

Qualitative LLM-as-a-Judge Evaluation:

We utilized **Google Gemini** as an automated **LLM-as-a-Judge** to evaluate the semantic, factual, and conversational quality of the generated scripts. All qualitative metrics are scored on a standardized scale of **0 to 10** (where **10** indicates optimal performance):

- **Completeness:** Measures how completely the generated script covers the source content.
- **Factual Accuracy:** Measures how faithfully the generated script follows the source PDF.
- **Concept Relevance:**Evaluates whether the dialogue remains focused on the source topic.
- **Analogy Accuracy:** Measures the correctness of real-world analogies and examples.
- **Non-Contradiction:** Confirms the script avoids statements that contradict the original source material.
- **Coherence Flow:** Evaluates the logical flow and transitions between dialogue turns.
- **Persona Consistency:** Assesses consistency in maintaining the Egyptian podcast speakers' personas.

Table 4.2: Model A - Qualitative LLM Evaluation Metrics

Evaluation Metric	Mean	Median	Min / Max
Persona Consistency	9.51	10.00	3.0 / 10.0
Concept Relevance	9.11	9.00	3.0 / 10.0
Factual Accuracy	8.99	10.00	1.0 / 10.0
Coherence Flow	8.58	9.00	3.0 / 10.0
Analogy Accuracy	8.50	9.00	0.0 / 10.0
Completeness	7.42	8.00	1.0 / 10.0

Table 4.3: Model B - Qualitative LLM Evaluation Metrics

Evaluation Metric	Mean	Median	Min / Max
Persona Consistency	9.14	10.00	2.0 / 10.0
Concept Relevance	8.71	9.00	1.0 / 10.0
Factual Accuracy	9.12	10.00	1.0 / 10.0
Coherence Flow	8.52	9.00	2.0 / 10.0
Analogy Accuracy	7.58	9.00	0.0 / 10.0
Completeness	5.62	6.00	1.0 / 10.0

Conclusion and Final Model Selection:

The comparative evaluation revealed a distinct trade-off between dialectal naturalness and technical control. **Model A (Iteration 1)** achieved superior scores in **Persona Consistency (9.51/10)**, **Analogy Accuracy (8.50/10)**, and **Completeness (7.42/10)**, resulting in a highly engaging and natural podcast flow. Conversely, **Model B (Iteration 2)** yielded higher **Factual Accuracy (9.12/10)**, but showed lower **Persona Consistency (9.14/10)** and **Completeness (5.62/10).**

Ultimately, **Model A** was selected for the final **Doc2Pod** pipeline. For an educational podcast targeting listener retention, the authentic Egyptian dialect, natural analogies, and comprehensive coverage provided by **Model A** outweigh the marginal technical gains of **Model B**. **Model A** represents the optimal balance for this use case, delivering high **Factual Accuracy (8.99/10)** alongside the conversational fluency required to maintain listener engagement.

4.4.3 Speech Synthesis (TTS) Acoustic Performance

The final empirical test evaluated the quality, accuracy, and naturalness of the custom fine-tuned Text-to-Speech acoustic model when handling the complex code-switching of English technical terms and Egyptian Arabic.

A. Phonetic Accuracy: Word Error Rate (WER)

- **The Core Benchmark (29.91%):**

  The synthesized audio was subjected to a Word Error Rate (WER) analysis. The engine achieved a baseline WER of 29.91%. While higher than standard monolingual TTS systems, this metric highlights the extreme phonetic complexity of forced bilingual Code-Switching. The engine must dynamically map English Latin characters (like "Data Structure") seamlessly into an Arabic phonological flow without prior phonetic transliteration.

B. Acoustic Naturalness: Mean Opinion Score (MOS)

To evaluate the human perception of the generated voices, a subjective Mean Opinion Score (MOS) study was conducted. A total of **139 independent participants** evaluated 10 synthesized audio samples on a standard 5-point scale (where 1 = Very Unnatural, and 5 = Very Natural).

MOS Conclusion:

The system achieved an **overall average MOS of 2.97**. However, the significant variance between samples (peaking at a highly natural 4.22 on Sample 1) indicates that while the base diffusion architecture is capable of producing near-human conversational quality, specific sequences of dense English technical terms currently induce minor prosodic robotic artifacts, providing a clear roadmap for future fine-tuning iterations.

5- User Manual
System Operation & Installation Guide

5.1 Chapter Overview

This chapter provides a complete guide for installing, configuring, and operating Doc2Pod. It is structured into two major sections: the Installation Guide (Section 5.2) and the User Operation Manual (Section 5.3).

Doc2Pod is composed of three interconnected components:

- Angular 21 Frontend — the web interface.
- ASP.NET Core 9 Backend — REST API handling authentication, document management, and podcast orchestration.
- Python AI Microservice (Google Colab) — performs OCR, RAG-based script generation, and Egyptian-dialect TTS synthesis.

Source Repositories:

- Frontend: https://github.com/Mohammed-Atef2004/Doc2Pod
- Backend: https://github.com/Mohammed-Atef2004/Doc2Pod-Backend
- AI Services: https://github.com/Mohammed-Atef2004/Doc2Pod-Services

5.2 Installation Guide

This section describes how to install and configure all software components required to run Doc2Pod. Follow the subsections in order.

5.2.1 System Prerequisites

The following software must be installed on the host machine before proceeding:

- Node.js v20 LTS or later — https://nodejs.org
- Angular CLI v21 — install via: npm install -g @angular/cli
- .NET SDK Version 9.0 — https://dotnet.microsoft.com/download
- SQL Server 2019 or later (Express edition is sufficient for development)
- Git Any recent version — https://git-scm.com

- Google Account Required to run the AI microservice on Google Colab
- Supabase Account Free tier is sufficient — https://supabase.com
- Gmail Account Used by the backend SMTP service for transactional emails

Note: The Python AI microservice runs entirely on Google Colab. No local Python installation is required. FFmpeg is used internally by the Colab pipeline for audio compression; it is installed automatically within the Colab environment.

5.2.2 Installing the Frontend

1. Clone and Install

```shell
git clone https://github.com/Mohammed-Atef2004/Doc2Pod.git
cd Doc2Pod
npm install
```

2. Start Development Server

```shell
ng serve
```

The application is then accessible at http://localhost:4200/

3. Production Build

```shell
ng build
```

5.2.3 Installing the Backend

1. Clone and Restore Packages

```shell
git clone
https://github.com/Mohammed-Atef2004/Doc2Pod-Backend.git
cd Doc2Pod-Backend
dotnet restore
```

2. Configure User Secrets

Sensitive credentials are stored using .NET User Secrets. Run the following from inside the API/ folder:

```shell
dotnet user-secrets set
"ConnectionStrings:DefaultConnection"
```

```
"Server=YOUR_SERVER;Database=Doc2Pod;Trusted_Connection=Tru
e;"
dotnet user-secrets set "Jwt:SecretKey"
"your-secret-key-minimum-32-characters"
dotnet user-secrets set "Jwt:Issuer" "doc2pod"
dotnet user-secrets set "Jwt:Audience" "doc2pod-users"
dotnet user-secrets set "EmailSettings:Email"
"your-gmail@gmail.com"
dotnet user-secrets set "EmailSettings:Password"
"your-gmail-app-password"
```

The following non-secret values are read from appsettings.json:

- Jwt:AccessTokenMinutes: 15
- Jwt:RefreshTokenDays: 7
- EmailSettings:Host: smtp.gmail.com
- EmailSettings:Port: 587
- ApiSettings:BaseUrl: http://localhost:5022 (update for production)
- PythonService:BaseUrl: Cloudflare tunnel URL — obtained in Section 5.2.4 Step 4

3. Set Up SQL Server Database

- Install SQL Server and ensure the service is running.
- Create an empty database named Doc2Pod.
- Update the connection string in user-secrets to point to the new database.
- Apply EF Core migrations to create all tables:

Shell

```
dotnet ef database update --project ../Infrastructure
--startup-project .
```

This creates the Users, Documents, Podcasts, and AuditLogs tables.

4. Run the API

Shell

```
dotnet run
```

The API is available at http://localhost:5022 and the Swagger UI at http://localhost:5022/swagger.

5.2.4 Setting Up the AI Microservice (Google Colab)

The AI pipeline runs as a FastAPI server inside a GPU-enabled Google Colab session. It handles OCR, LLM-based script generation, and Egyptian-dialect TTS synthesis. A Cloudflare Tunnel exposes the server over a public HTTPS URL that the .NET backend uses to communicate with it.
Uploaded PDFs, generated scripts, and generated audio files are stored in Supabase Storage using three dedicated buckets. These buckets must be created before running the notebook.

Step 1 — Create Supabase Storage Buckets

Create a free account at supabase.com, then create three storage buckets with the exact names below:

Bucket Name	Purpose
PDFs	Stores uploaded PDF documents
Scripts	Stores generated podcast scripts
Podcasts	Stores final MP3 audio files

From the Supabase project settings, copy the **Project URL** and the **service_role API key**. These are needed in the next step.

Step 2 — Add Supabase Credentials to Colab Secrets

Open the Colab notebook and click the key icon in the left sidebar. Add two secrets:

- SUPABASE_URL — your Supabase project URL
- SUPABASE_KEY — your Supabase service_role API key

Step 3 — Configure the Colab Runtime

Set the runtime type to GPU: **Runtime → Change runtime type → T4 GPU**. Then run all cells sequentially from Step 0 (Logging Setup) onwards. Model loading and environment setup typically completes within several minutes on a fresh runtime.

Step 4 — Copy the Cloudflare Tunnel URL

In the output of the cell labelled **Step 17: Launch Doc2Pod Server**, a line will appear in the following format:

```
None
Cloudflare API Link: https://xxxxx.trycloudflare.com
```

Copy this full URL.

Step 5 — Connect the Backend to the AI Service

Open appsettings.json in the .NET API project and set the PythonService:BaseUrl value to the Cloudflare URL copied in the previous step, then restart the API. The backend will now forward all podcast generation requests to the live Colab server.

Note: The Cloudflare tunnel URL changes every time a new Colab session starts. The PythonService:BaseUrl must be updated each session.

Step 6 — Verify the Connection

Submit a podcast generation request from the frontend. In the Colab Step 17 output cell, pipeline log messages will appear in real time, confirming that the .NET backend and Python AI service are communicating correctly.

5.3 User Operation Manual

This section provides a step-by-step walkthrough of every user-facing feature, from first launch to playing a generated podcast.

5.3.1 Landing Page

When the application is accessed at http://localhost:4200/, the Doc2Pod landing page is displayed, as shown in Figure 5.1.

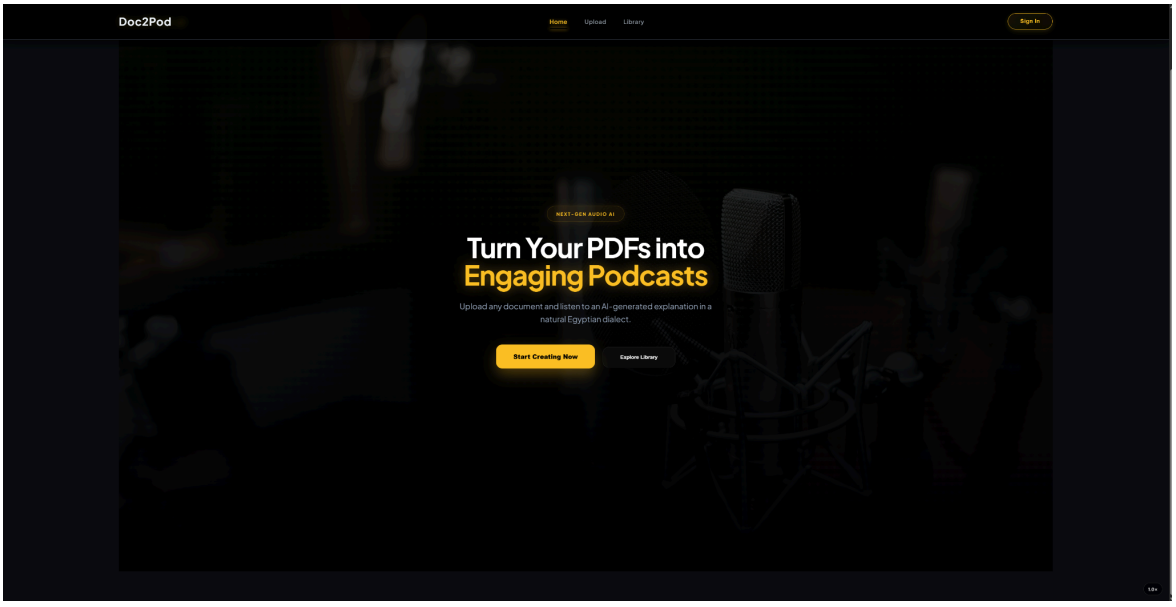

Figure 5.1 Landing page

The navigation bar contains links to Home, Upload, and Library, along with a Sign In button. The hero section presents two primary calls to action: Start Creating Now (navigates to the Upload page) and Explore Library (navigates to the podcast library).

5.3.2 Creating a New Account

To register, click Sign In in the navigation bar, then click 'Create one now' on the login screen. The registration form shown in Figure 5.2 is displayed.

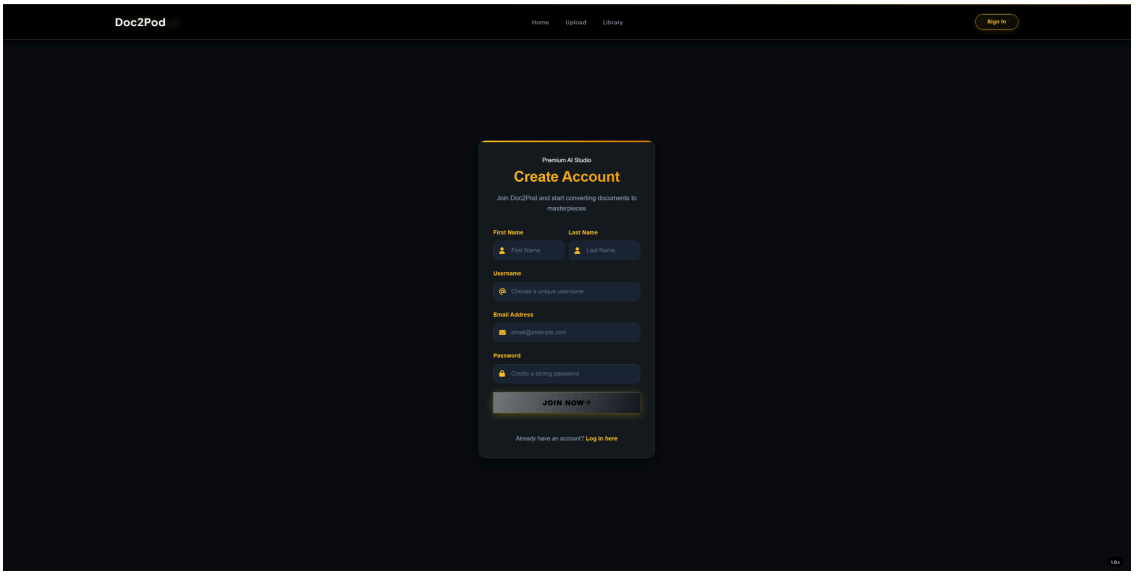

Figure 5.2 Account registration interface

The registration form requires: First Name, Last Name, a unique Username, Email Address, and a Password. Figure 5.3 shows the form completed with sample data.

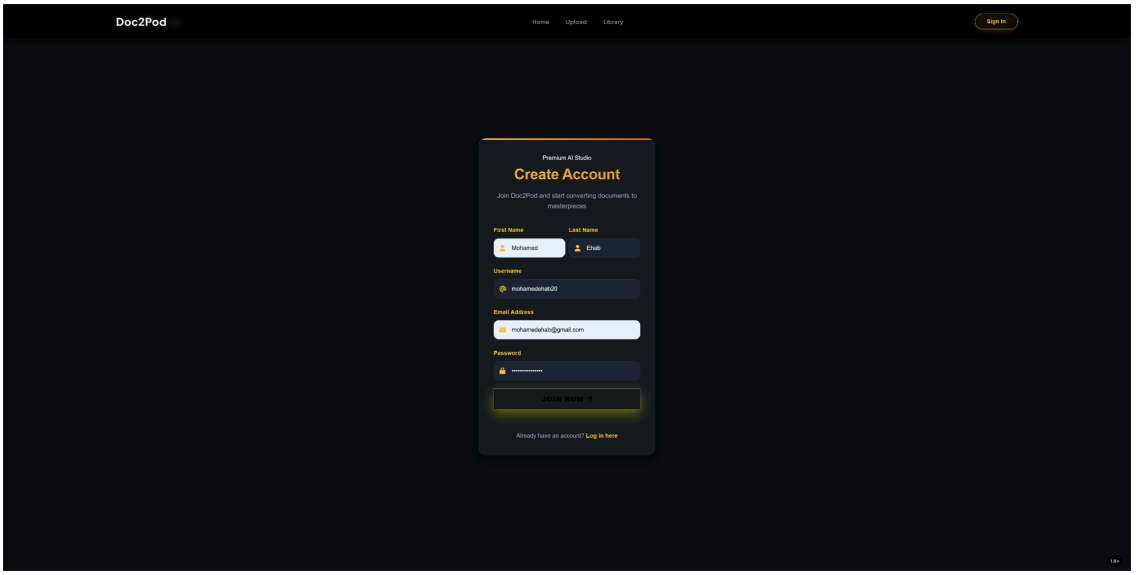

Figure 5.3 Registration form with sample data

On clicking JOIN NOW, the system creates the account and sends an email verification message to the provided address.

5.3.3 Email Verification

A confirmation email from Doc2Pod is delivered to the registered inbox, as shown in Figure 5.4. Clicking VERIFY MY ACCOUNT activates the account and redirects the user to the application.

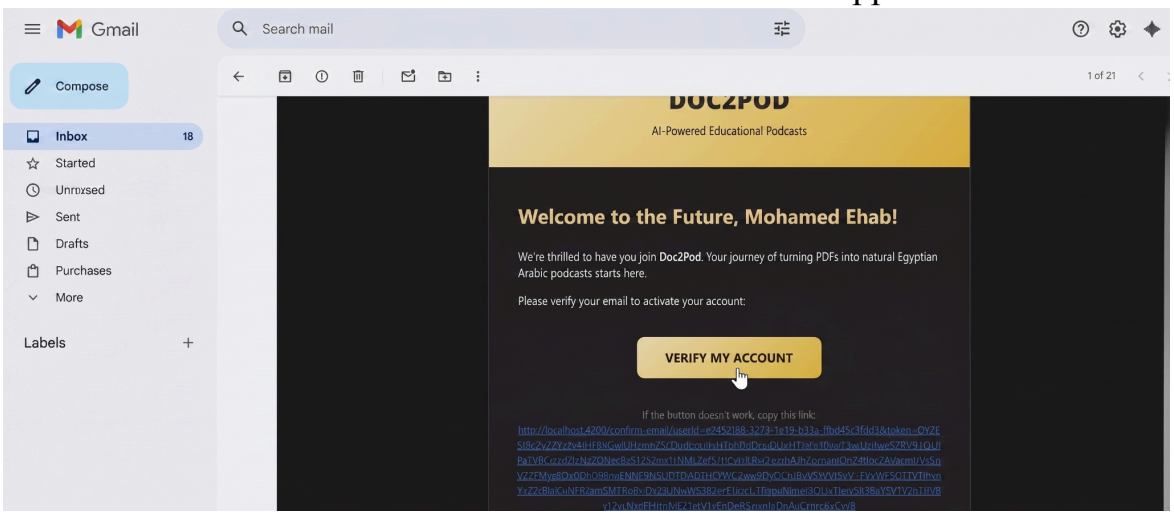

Figure 5.4 Email verification notification

The application then displays the confirmation screen shown in Figure 5.5, after which the user is directed to the login page.

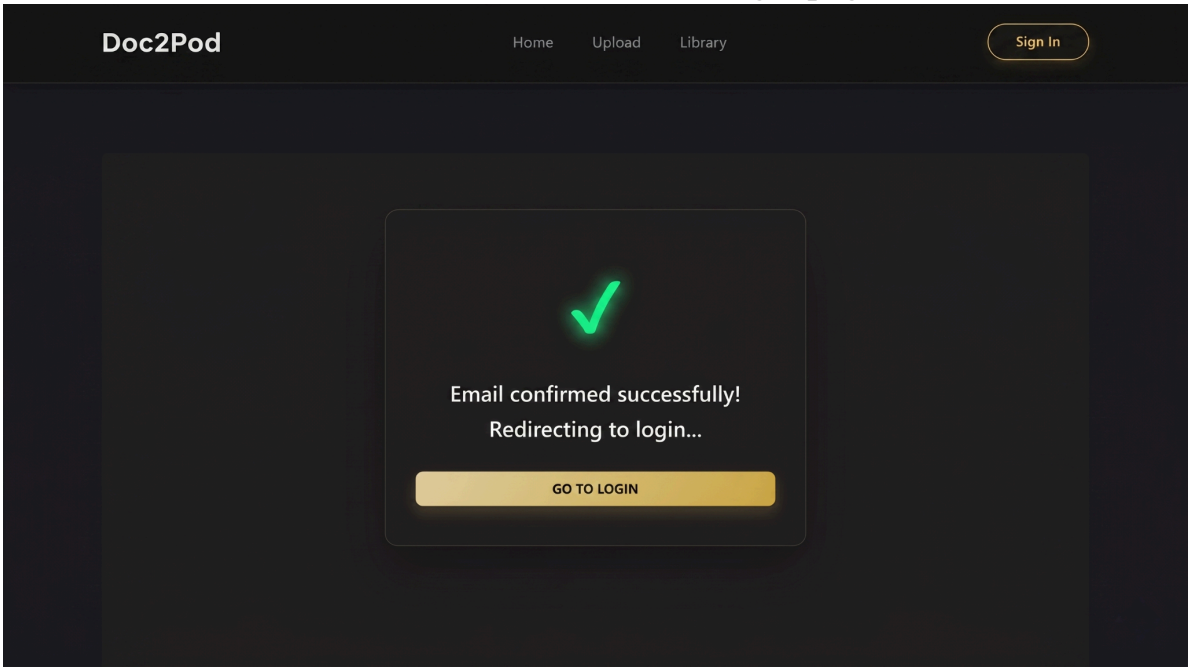

Figure 5.5 Account confirmation screen

5.3.4 Signing In

Users with a verified account can log in via the Sign In button. The login form is shown in Figure 5.6.

Upon the submission of valid account details and selecting **Sign In**, Following this, the interface automatically transitions to the home screen.

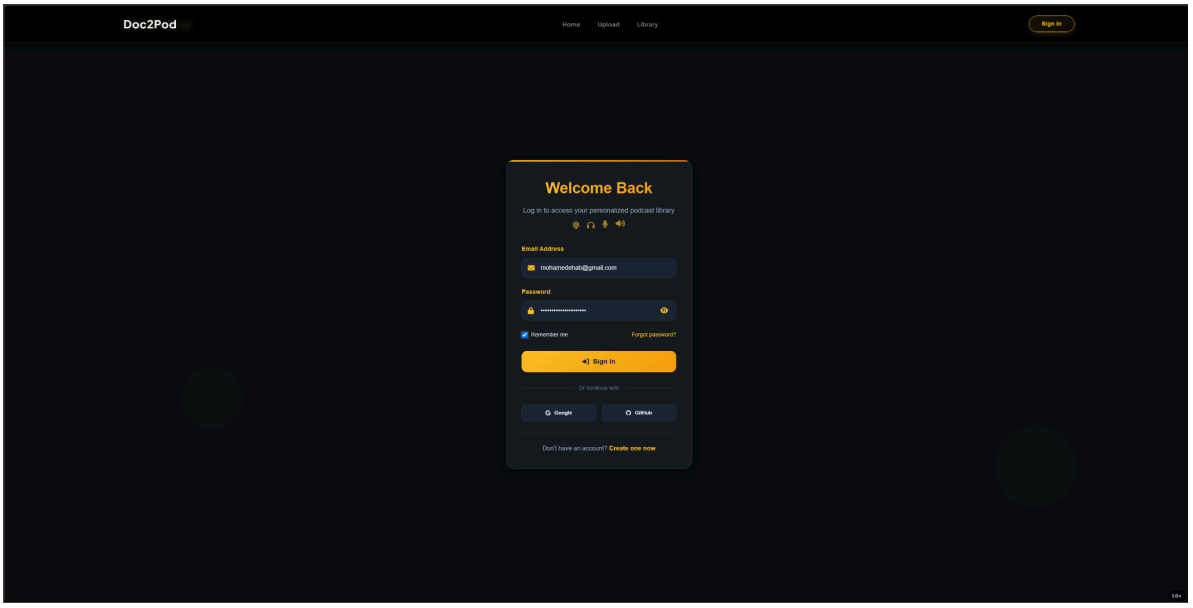

Figure 5.6 User authentication interface

5.3.5 Uploading a Document & Configuring Generation

Clicking Upload in the navigation bar opens the Transform PDF to Podcast page as shown in Figure 5.7. Two document selection options are available:

- Upload New PDF — opens a file picker; the selected file is uploaded to Supabase Storage and a documentId is returned by the backend.
- Choose Existing PDF — allows selection of a previously uploaded document, avoiding duplicate uploads.

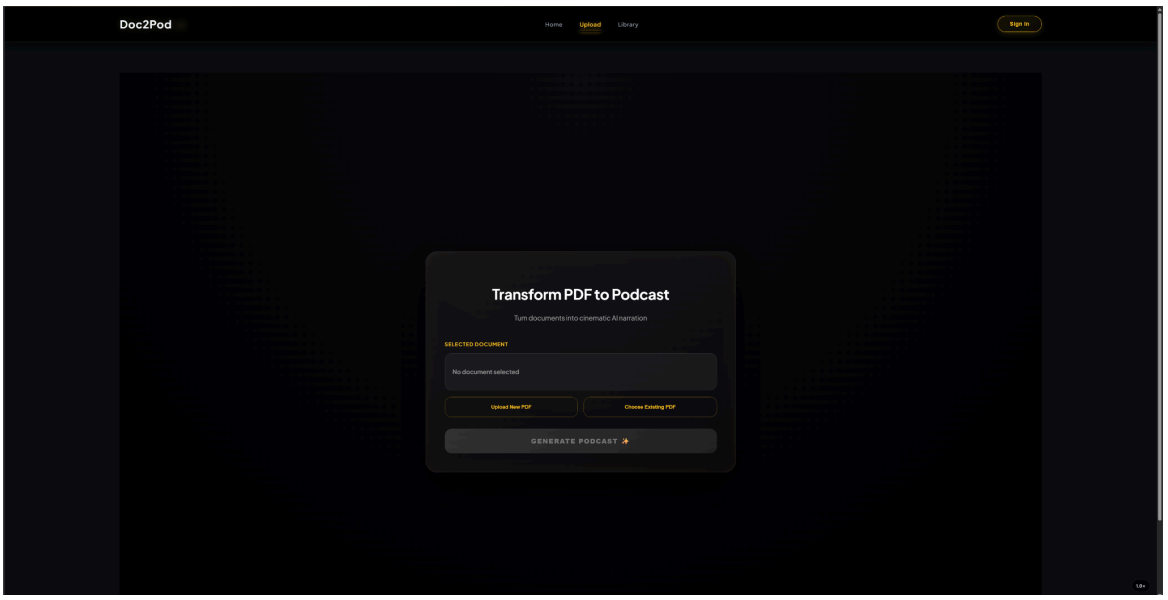

Figure 5.7 Document-to-podcast transformation interface

Once a file is designated, the interface reveals additional settings shown in Figure 5.8. Within this panel, users must define their preferred **Voice Generation Model (VibeVoice or Gemini API)** and **Generation Mode** before initiating the process.

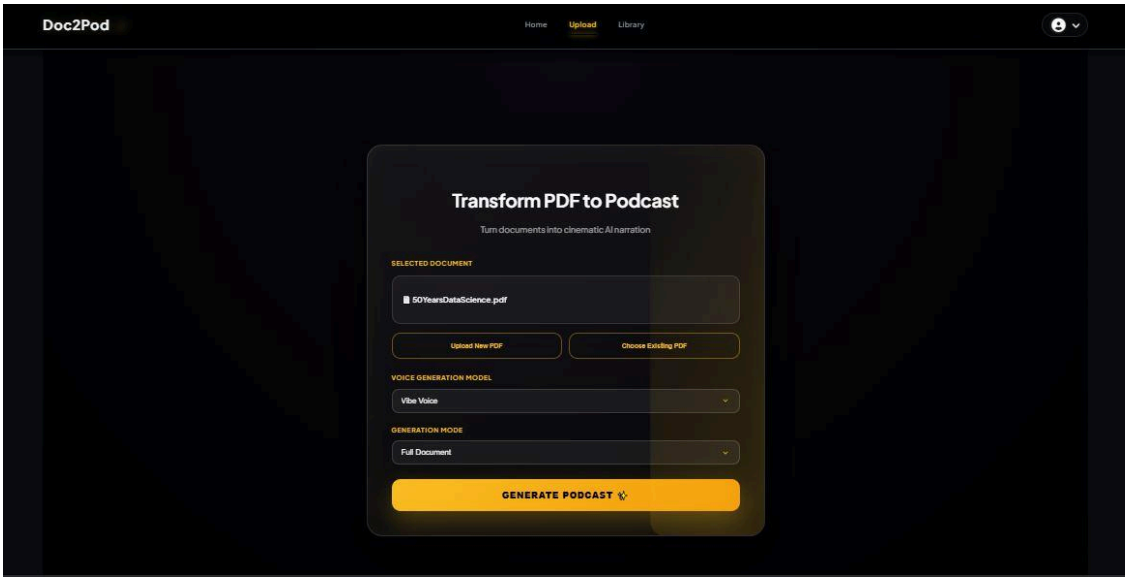

Figure 5.8 Podcast generation configuration settings

Three generation modes are available from the Generation Mode dropdown.

Generation Mode	Description	Parameters
Topic Focus	Generates a podcast focused on a specific question or topic. The AI retrieves the most relevant content from the document using RAG.	Topic
Topic + Page Range	Combines topic-based retrieval with a page range constraint. Suited when relevant content is known to lie in a specific section.	Topic, Start Page, End Page
Full Document	Generates a comprehensive summary podcast covering the entire document.	None

5.3.6 Podcast Generation

After configuring the parameters, the user clicks GENERATE PODCAST. The frontend dispatches the request to the .NET backend, which forwards it to the Python AI microservice. A loading screen is displayed in Figure 5.9 while the system polls for completion.

Now You have to wait until the Generation Completes and the successful message pops up

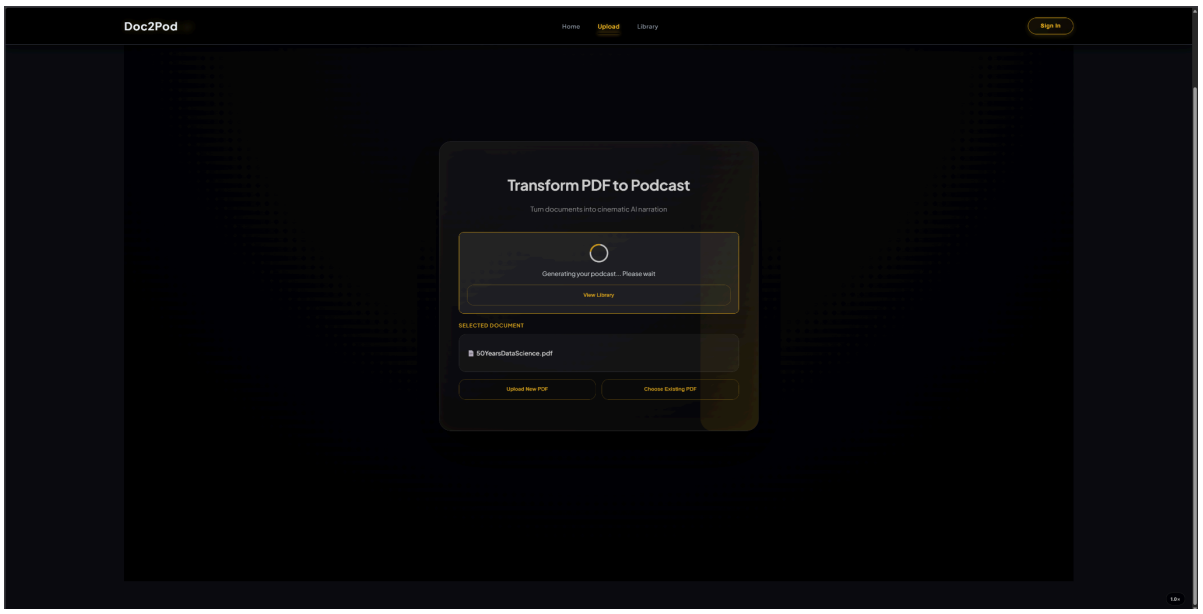

Figure 5.9 Podcast generation loading screen

5.3.7 Podcast Library

The Library page (Figure 5.10) acts as a persistent repository of all podcasts generated by the authenticated user. It is accessible from the Library link in the navigation bar.

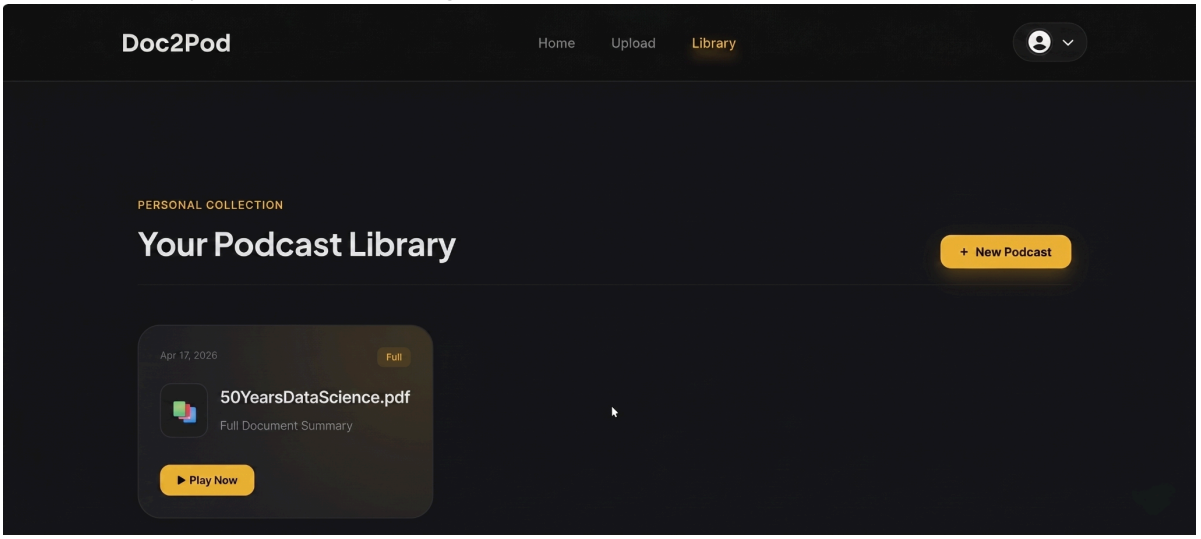

Figure 5.10 Podcast library interface

Each card displays the document name, generation date, and mode label. Clicking + New Podcast returns to the Upload page. To play a podcast from a previous session, the user simply navigates to the Library and selects the relevant card — no regeneration is required, as the audio is permanently stored in Supabase.

5.3.8 Playing a Podcast

Clicking Play Now opens the audio player overlay, shown in Figure 5.11, which streams the MP3 directly from Supabase Storage.

Figure 5.11 Podcast audio playback interface

6- Conclusion and Future Work

6.1 Conclusion

In this project, we developed **Doc2Pod**, an AI-powered educational system that transforms Computer Science PDF documents into long-form podcasts in Egyptian Arabic while preserving English technical terminology. The system was designed to make technical educational content more accessible and engaging by converting traditional text-based materials into conversational audio experiences.

The proposed solution combines multimodal document extraction, semantic content organization, retrieval-augmented generation (RAG), dynamic prompt engineering, and speech synthesis within a unified pipeline. This architecture enables the extraction and organization of educational concepts, retrieval of relevant information, generation of coherent podcast-style dialogues, and synthesis of spoken audio suitable for educational content delivery.

Key Achievements

- Developed a complete end-to-end pipeline for converting Computer Science PDFs into educational podcasts.
- Integrated **PaddleOCR-VL** and **Qwen-VL** to extract and interpret both textual and visual educational content.
- Implemented concept-based chunking, semantic chunk merging, and page-aligned subchunking to preserve document structure and contextual continuity.
- Built a retrieval pipeline that achieved **85.1% retrieval precision**, enabling accurate selection of educational content during podcast generation.
- Developed a state-aware generation framework capable of maintaining coherent technical discussions exceeding **30 minutes** in duration.
- Fine-tuned **Qwen3-4B-Instruct** and **VibeVoice-1.5B** to support Egyptian Arabic educational podcast generation while preserving English technical terminology.
- Achieved a **mean factual accuracy score of 8.99/10** and a **mean persona consistency score of 9.51/10**, demonstrating strong educational reliability and conversational quality.

Overall, the obtained results demonstrate the effectiveness of combining multimodal understanding, retrieval systems, large language models, and speech

synthesis to transform traditional educational resources into engaging audio-based learning experiences. The developed system provides a strong foundation for future AI-driven educational content generation and podcast-based learning applications.

6.2 Future Work

Several directions can be explored to further enhance and extend the capabilities of the **Doc2Pod** system:

1. Document Format Expansion

- Extend support to additional educational formats beyond PDFs, such as presentation slides,etc.

2. Speech Synthesis and Language Expansion

- Improve naturalness and expressiveness of generated speech, especially in long-form scenarios.
- Enhance prosody modeling and reduce robotic artifacts in extended utterances.
- Further optimize segmentation strategies to ensure smoother and more coherent audio generation.
- Extend speech capabilities to support additional Arabic dialects to improve accessibility across the MENA region.

3. Cross-Domain Expansion

- Extend the system beyond Computer Science to support additional academic disciplines.
- Adapt the framework for scientific, technical, and educational content across multiple domains.

References

Cui, C., Sun, T., Liang, S., Gao, T., Zhang, Z., Liu, J., Wang, X., Zhou, C., Liu, H., Lin, M. and Zhang, Y., 2025. Paddleocr-vl: Boosting multilingual document parsing via a 0.9 b ultra-compact vision-language model. *arXiv preprint arXiv:2510.14528*.

Blecher, L., Cucurull Preixens, G., Scialom, T. and Stojnic, R., 2024, May. Nougat: Neural optical understanding for academic documents. In *International Conference on Learning Representations* (Vol. 2024, pp. 37646-37663).

Guo, Z., Ren, X., Xu, L., Zhang, J. and Huang, C., 2025. Rag-anything: All-in-one rag framework. *arXiv preprint arXiv:2510.12323*.

Han, S., Xia, P., Zhang, R., Sun, T., Li, Y., Zhu, H. and Yao, H., 2025. Mdocagent: A multi-modal multi-agent framework for document understanding. *arXiv preprint arXiv:2503.13964*.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C. and Zheng, C., 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Team, G., Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M.S., Love, J. and Tafti, P., 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A. and Yang, A., 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Ren, Y., Liu, J. and Zhao, Z., 2021. Portaspeech: Portable and high-quality generative text-to-speech. *Advances in Neural Information Processing Systems*, *34*, pp.13963-13974.

Casanova, E., Weber, J., Shulby, C.D., Junior, A.C., Gölge, E. and Ponti, M.A., 2022, June. Yourtts: Towards zero-shot multi-speaker tts and zero-shot voice conversion for everyone. In *International conference on machine learning* (pp. 2709-2720). PMLR.

Peng, Z., Yu, J., Wang, W., Chang, Y., Sun, Y., Dong, L., Zhu, Y., Xu, W., Bao, H., Wang, Z. and Huang, S., 2025. Vibevoice technical report. *arXiv preprint arXiv:2508.19205*.

Chen, Y., Niu, Z., Ma, Z., Deng, K., Wang, C., JianZhao, J., Yu, K. and Chen, X., 2025, July. F5-tts: A fairytaler that fakes fluent and faithful speech with flow matching. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 6255-6271).

Xiao, Y., He, L., Guo, H., Xie, F.L. and Lee, T., 2025, July. Podagent: A comprehensive framework for podcast generation. In *Findings of the Association for Computational Linguistics: ACL 2025* (pp. 23923-23937).

Ju, Z., Yang, D., Kai, S., Leng, Y., Wang, Z., Liu, S., Zhou, X., Qin, T., Li, X., Yu, J. and Tan, X., 2026. Mooncast: High-quality zero-shot podcast generation. *Advances in Neural Information Processing Systems*, *38*, pp.6283-6317.

Biswas, A., AI-Driven Podcast Generation from Technical Documents Using Retrieval-Augmented Generation and Multi-Speaker TTS.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. *Advances in neural information processing systems*, *30*.

Kim, H. and Kim, B.H., 2025, July. Nexussum: Hierarchical llm agents for long-form narrative summarization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 10120-10157).

Toshevska, M. and Gievska, S., 2021. A review of text style transfer using deep learning. *IEEE Transactions on Artificial Intelligence*, *3*(5), pp.669-684.

Lodagala, V.S., Alkanhal, L., Izham, D., Mehta, S., Chowdhury, S.A., Makki, A., Hussein, H.S., Henter, G.E. and Ali, A., 2025. SawtArabi: A Benchmark Corpus for Arabic TTS. Standard, Dialectal and Code-Switching. In *Proc. Interspeech 2025* (pp. 4793-4797).

Al-Sabbagh, R., 2024. ArzEn-MultiGenre: An aligned parallel dataset of Egyptian Arabic song lyrics, novels, and subtitles, with English translations. *Data in Brief*, *54*, p.110271.

Alyafeai, Z. and Al-Shaibani, M.S., 2020, November. ARBML: Democratizing Arabic natural language processing Tools. In *Proceedings of second workshop for NLP open source software (NLP-OSS)* (pp. 8-13).

Xiao, Y., Xue, L., He, L., Chen, X., Chiu, A.Y.F., Tian, W., Zhang, S., Kong, Q., Zhu, X., Xue, W. and Lee, T., 2025. PodEval: A Multimodal Evaluation Framework for Podcast Audio Generation. *arXiv preprint arXiv:2510.00485*.

Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. and Zhang, H., 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, *36*, pp.46595-46623.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R. and Zhu, C., 2023, December. G-eval: NLG evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 2511-2522).

Bsharat, S.M., Myrzakhan, A. and Shen, Z., 2023. Principled instructions are all you need for questioning llama-1/2, gpt-3.5/4. *arXiv preprint arXiv:2312.16171*.